arXiv:2605.10614v1 [cs.AI] 11 May 2026

PRISM: Generation-Time Detection and Mitigation of Secret Leakage in Multi-Agent LLM Pipelines

Riya Tapwal
School of Computing and Electrical Engineering
Indian Institute of Technology, Mandi (IIT)
riya@iitmandi.ac.in

Abhishek Kumar
The Alan Turing Institute, London
akumar@turing.ac.uk

Carsten Maple
University of Warwick
cm@warwick.ac.uk

Abstract

Multi-agent LLM systems introduce a security risk in which sensitive information accessed by one agent can propagate through shared context and reappear in downstream outputs, even without explicit adversarial intent. We formalise this phenomenon as *propagation amplification*, where leakage risk increases across agent boundaries as sensitive content is repeatedly exposed to downstream generators. Existing defences, including prompt-based safeguards, static pattern matching, and LLM-as-judge filtering, are not designed for this setting: they either operate after generation, rely primarily on surface-form patterns, or add substantial latency without modelling the generation process itself. To resolve these issues, we propose PRISM, a real-time defence that treats credential leakage as a sequential risk accumulation problem during generation. At each decoding step, PRISM combines 16 signals spanning lexical, structural, information-theoretic, behavioural, and contextual features into a calibrated risk score, enabling per-token intervention through green, yellow, and red risk zones. Our central observation is that credential reproduction is often preceded by a measurable shift in generation dynamics, characterised by entropy collapse and increasing logit concentration. When combined with text-structural cues such as identifier-pattern detection, these temporal signals provide an early warning of leakage before a secret is fully reconstructed. Across a 2,000-task adversarial benchmark covering 13 attack categories and three pressure levels in a heterogeneous four-agent pipeline, PRISM achieves $F_1 = 0.832$ with precision $= 1.000$ and recall $= 0.712$, while producing **no observed leakage** on our benchmark (0.0% task-level leak rate) and preserving output utility of 0.893. It substantially outperforms the strongest baseline, Span Tagger, which achieves $F_1 = 0.719$ with a 15.0% task-level leak rate.

1 Introduction

Multi-agent LLM systems increasingly rely on shared context and tool access to coordinate complex software-engineering workflows. However, this collaboration creates a security risk: sensitive information retrieved by one agent may be unintentionally exposed to downstream agents and reproduced in later outputs. For example, a multi-agent assistant tasked with debugging a production software system may invoke `read_file(".env.production")` and retrieve a secret key. Although the key is not needed to solve the task, it can enter the shared context and subsequently appear in intermediate summaries, generated code, execution logs, or the final user-facing response. This failure does not require an explicit adversarial prompt; ordinary agent collaboration is sufficient. It

exposes a structural vulnerability in multi-agent LLM systems: once sensitive information enters a shared context, it can propagate across agents and re-emerge downstream.

Recent advances in multi-agent LLM frameworks, including AutoGen [Wu et al., 2024], CAMEL [Li et al., 2023], ChatDev [Qian et al., 2024], and MetaGPT [Hong et al., 2023], have enabled collaborative workflows that combine planning, retrieval, reasoning, code generation, and execution. These systems differ from single-model deployments in a critical way: information retrieved or generated by one agent is often appended to a shared, evolving context and consumed by subsequent agents. As a result, sensitive information accessed at one stage may persist across the pipeline and be reproduced by later agents, even when it is no longer relevant to the task. We formalise this phenomenon as *propagation amplification*. For a pipeline of K agents $\{A_1, \ldots, A_K\}$ operating over shared context $\mathcal{C}$, let a secret s be retrieved at step i. The probability that s is leaked by at least one downstream agent can be written as

$$P_{\text{leak}}(s) = 1 - \prod_{j=i+1}^{K} \big(1 - p_j(s \mid \mathcal{C}_j)\big), \tag{1}$$

where $p_j(s \mid \mathcal{C}_j)$ denotes the probability that agent A_j reproduces s given its context. Under the mild lower-bound assumption $\delta = \min_j p_j(s \mid \mathcal{C}_j) > 0$, this gives $P_{\text{leak}}(s) \geq 1 - (1-\delta)^{K-i}$.. Thus, even small per-agent reproduction probabilities can accumulate into substantial leakage risk as pipeline depth increases. In our undefended four-agent pipeline, 71.2% of tasks produce at least one secret leak, with similar rates across adversarial pressure levels: 70.4% under low pressure, 70.1% under medium pressure, and 73.5% under high pressure. This suggests that leakage is not only a prompt-level attack phenomenon, but also a structural consequence of shared-context agent composition. This risk is consistent with prior evidence on credential exposure and LLM privacy failures. Large-scale studies have shown that secrets frequently appear in real-world code repositories [Meli et al., 2019], while recent benchmarks identify persistent leakage risks in LLM-based systems [Zhou et al., 2025]. Broader surveys on LLM security and privacy [Aguilera-Martínez and Berzal, 2025, Kibriya et al., 2024, Chen et al., 2025] also highlight sensitive information exposure as a continuing challenge, particularly in systems with complex interaction patterns. Multi-agent systems further amplify this concern: recent work such as AgentLeak [Yagoubi et al., 2026] shows that internal communication channels can expose sensitive information beyond final user-facing outputs. Existing defences are not well matched to this setting. Prompt-based safeguards attempt to constrain model behaviour through instructions [Schulhoff et al., 2023, Wei et al., 2023, Mehrotra et al., 2024], but can degrade under multi-hop reasoning and adversarial prompting. Static scanners such as detect-secrets [Yelp, 2018] and TruffleHog [Truffle Security Co., 2023] rely primarily on surface-level patterns and are not designed to capture contextual reuse or partial reconstruction of secrets in free-form LLM outputs. LLM-as-judge methods [Zheng et al., 2023] and guardrail systems such as Llama Guard [Inan et al., 2023] and LlamaFirewall [Chennabasappa et al., 2025] can provide useful filtering, but they typically operate after generation and target broad safety categories rather than credential-specific leakage. In contrast, secret leakage in multi-agent pipelines unfolds as a sequential process during generation, requiring defences that can intervene before sensitive content is fully reconstructed and propagated.

We propose PRISM (**P**redictive **R**isk **I**ntervention for **S**ecret **M**onitoring), a real-time defence that models credential leakage as a sequential risk accumulation process during generation. At each decoding step, PRISM aggregates 16 heterogeneous signals spanning lexical, structural, information-theoretic, behavioural, and contextual features into a calibrated risk score,$r_t = \sigma(\mathbf{w}^\top \mathbf{f}_t + b)$,, which supports per-token intervention through graduated risk zones during streaming generation. The key observation underlying PRISM is that credential reproduction is often preceded by a measurable shift in generation dynamics: as a model moves from open-ended generation toward deterministic reproduction of a structured secret, token-level entropy decreases and probability mass becomes increasingly concentrated. By combining these temporal signals with text-structural cues, such as identifier-pattern and credential-style features, PRISM detects leakage before a secret is fully reconstructed.

Contributions:

1. We formalise *propagation amplification* as a structural vulnerability in multi-agent LLM systems and provide a probabilistic model explaining how leakage risk compounds across agent boundaries.

2. We introduce PRISM, a real-time, multi-signal defence that performs per-token risk estimation and graduated intervention during generation, enabling low-latency mitigation of credential leakage.
3. We identify a complementary set of detection signals, including entropy collapse and logit concentration from temporal generation dynamics, together with identifier-pattern and credential-style structure from generated text, and show that neither signal class alone achieves the discriminative power of the full feature set.
4. We construct a 2,000-task benchmark for multi-agent secret leakage across 13 attack categories and three adversarial pressure levels, enabling large-scale empirical comparison across ten defence methods. On this benchmark, we observe that generation-time monitoring achieves $F_1 = 0.832$ with **no observed leakage** on our benchmark (0.0% task-level leak rate) and utility of 0.893, outperforming the strongest baselines, including Span Tagger ($F_1 = 0.719$, 15.0% task-level leak rate) and GBT Classifier ($F_1 = 0.684$, 19.2% task-level leak rate).

2 Related Work

Research on LLM security and secret leakage has expanded rapidly, with surveys such as Aguilera-Martínez and Berzal [2025] and Kibriya et al. [2024] highlighting the growing attack surface. We focus on work most relevant to credential leakage in agentic pipelines.

Credential leakage and static detection: Early work documented widespread credential exposure in public repositories [Meli et al., 2019], motivating static scanners such as detect-secrets [Yelp, 2018] and TruffleHog [Truffle Security Co., 2023], which rely primarily on entropy thresholds and regex-based patterns. Subsequent work has studied leakage in LLM-based systems, including LessLeak-Bench [Zhou et al., 2025] and prompt-based extraction of embedded secrets [Agarwal et al., 2024]. While these studies establish the prevalence of secret leakage, they largely operate on static code, benchmark-level data exposure, or single-model interactions, and therefore do not directly address leakage propagation in multi-agent pipelines. Related work on adversarial prompting further shows that LLM safeguards remain brittle: large-scale studies and benchmarks [Schulhoff et al., 2023, Wei et al., 2023, Chao et al., 2024] demonstrate that safety alignment can be bypassed through jailbreak attacks. Automated attack methods such as Tree-of-Attacks [Mehrotra et al., 2024] and universal adversarial suffixes [Zou et al., 2023] further illustrate the persistence of these failures, while recent work [Wang et al., 2025] confirms that prompt injection remains effective even against safety-trained models.

Safety classifiers and LLM-based filtering: LLM-as-judge frameworks [Zheng et al., 2023] and safety classifiers such as Llama Guard [Inan et al., 2023] and LlamaFirewall [Chennabasappa et al., 2025] apply filtering or moderation to generated content. Systems such as ControlNET [Yao et al., 2025] and NeMo Guardrails [Rebedea et al., 2023] extend this paradigm to more structured LLM workflows. However, these approaches primarily target broad content-safety categories rather than credential-specific leakage, and they typically operate after generation has already occurred. This makes them less suitable for multi-agent settings in which a secret may enter shared context and propagate before final-output filtering is applied. Code-specific defences [Kavian et al., 2024] highlight the need for domain-specific analysis, but do not model generation-time leakage dynamics.

Multi-agent systems and propagation: Multi-agent frameworks such as ChatDev [Qian et al., 2024], MetaGPT [Hong et al., 2023], AutoGen [Wu et al., 2024], CAMEL [Li et al., 2023], CrewAI [CrewAI, 2024], and HuggingGPT [Shen et al., 2023] enable collaborative workflows through inter-agent communication, tool use, and shared intermediate state. These capabilities also introduce new attack surfaces, since information produced or retrieved by one agent can become visible to downstream agents. Subsequent work explores agent capabilities and behaviours [Liu et al., 2024, Wang et al., 2023, Chen et al., 2024, Park et al., 2023], but generally does not treat credential leakage as a central threat model. AgentLeak [Yagoubi et al., 2026] provides closer evidence that sensitive information can appear in internal agent communications and leak across channels. While that work focuses on broader privacy leakage, our setting targets immediately exploitable credentials and emphasises intervention during generation, before sensitive content is fully reconstructed or propagated.

Static scanners and non-LLM detection: Static tools such as detect-secrets [Yelp, 2018], TruffleHog [Truffle Security Co., 2023], and Presidio [Microsoft, 2019] are designed for structured code, secrets, and PII detection. Applying them to free-form LLM outputs requires substantial recalibration because generated text differs from source code in token distribution, formatting regularity, and contextual reuse of sensitive content. Classical ML approaches, such as gradient-boosted classifiers and span taggers, provide useful baselines, but they operate on completed outputs and lack access to generation-time dynamics. Recent work also emphasises the need for structured evaluation of LLM risks [Maathuis and Chockalingam, 2024] and demonstrates persistent failures in safety-critical domains [Hung et al., 2023]. Surveys of privacy risks [Chen et al., 2025] further highlight the gap between broad content-safety moderation and credential-specific protection.

Overall, existing approaches remain limited for this setting because they are typically (i) static or post-hoc, (ii) designed primarily for single-model interactions, or (iii) focused on broad safety categories rather than credential-specific leakage. In contrast, PRISM provides a generation-time defence tailored to multi-agent pipelines, explicitly modelling propagation risk and enabling intervention before secrets are fully reconstructed.

3 Threat Model and Problem Formulation

3.1 System Model

We consider a sequential multi-agent pipeline $\mathcal{A} = \langle A_1, \ldots, A_K \rangle$, where each agent A_k may be instantiated from a different LLM and performs a functional role such as planning, retrieval, code generation, or execution. Agents communicate through an append-only shared context:

$$\mathcal{C}_k = \mathcal{C}_{k-1} \,\|\, o_{k-1}, \qquad \mathcal{C}_1 = (\texttt{task}, \texttt{ctx}),$$

where o_{k-1} denotes the output produced by agent A_{k-1} and $\|$ denotes context concatenation. Each agent has access to a tool set $\mathcal{G}$, such as `read_file`, `db_query`, and `api_call`, connected to a shared repository $\mathcal{R}$ containing secrets $\mathcal{S} = \{s_1, \ldots, s_M\}$. If a secret is retrieved or generated by an upstream agent, it may be appended to the shared context and thereby become available to all downstream agents.

3.2 Adversary Model

We assume a realistic black-box adversary who controls only the input prompt `task` and has no access to model parameters, tool implementations, or repository contents. This captures a malicious or compromised user interacting with an otherwise trusted multi-agent system. Leakage may arise in three forms: *direct leakage*, where an agent retrieves and emits a secret; *propagation leakage*, where downstream agents reproduce a secret previously introduced into the shared context; and *adversarial extraction*, where prompt-level manipulation increases the likelihood of disclosure through coercive cues. We model adversarial pressure using a transformation $\phi(t, p)$, where $p \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$ controls the strength of such cues, including urgency, authority, or explicit requests for sensitive information.

3.3 Leakage Process

Definition 1 (Secret Leakage)**.** *A task t produces leakage if the final output o_K contains any secret $s \in \mathcal{S}$:*

$$\text{LEAK}(t) = \mathbb{1}[\exists\, s \in \mathcal{S} : s \sqsubseteq o_K].$$

Leakage is sequential in nature. Once a secret enters the shared context, it may be reproduced by one or more downstream agents. If a secret s is introduced at step i, its probability of being leaked by at least one downstream agent is

$$P_{\text{leak}}(s) = 1 - \prod_{j=i+1}^{K} (1 - p_j(s \mid \mathcal{C}_j)),$$

where $p_j(s \mid \mathcal{C}_j)$ denotes the probability that agent A_j reproduces s given its context. Under the mild lower-bound assumption $p_j(s \mid \mathcal{C}_j) \geq \delta > 0$, this yields

$$P_{\text{leak}}(s) \geq 1 - (1 - \delta)^{K-i}.$$

Thus, even small per-agent reproduction probabilities can accumulate into substantial overall leakage risk as pipeline depth increases. We refer to this compounding effect as *propagation amplification*.

Remark on independence: The product formulation above corresponds to the conditional-independence case, where downstream reproduction events are treated as independent conditioned on their contexts. In practice, this assumption is conservative for append-only multi-agent pipelines. If agent A_j reproduces s, the secret becomes part of $\mathcal{C}_{j+1}$, which can increase $p_{j+1}(s \mid \mathcal{C}_{j+1})$ for subsequent agents. Such positive dependence strengthens the qualitative conclusion that leakage risk is non-decreasing with pipeline depth. Equivalently, the Bonferroni union bound,

$$P_{\text{leak}}(s) \leq \sum_j p_j(s \mid \mathcal{C}_j),$$

and corresponding inclusion–exclusion terms support the same intuition: as additional downstream agents are exposed to the secret, the opportunity for reproduction increases. Empirically, we validate this effect by varying pipeline depth $K \in \{1, 2, 3, 4\}$ and observing a monotone increase in task-level leak rate.

3.4 Defence Objective

A defence $\mathcal{D}$ maps each intermediate output o_k to a filtered output $\hat{o}_k$. The objective is to maximise leakage detection while limiting unnecessary modification of benign outputs:

$$\max_{\mathcal{D}} \; F_1(\mathcal{D}, \mathcal{S}) \quad \text{s.t.} \quad \text{OverBlock}(\mathcal{D}) \leq \epsilon.$$

Here, $F_1(\mathcal{D}, \mathcal{S})$ measures detection performance over protected secrets, and $\text{OverBlock}(\mathcal{D})$ denotes the fraction of benign tasks whose outputs are unnecessarily modified. We define task-level utility as

$$\mathcal{U}(\mathcal{D}) = \frac{|\{t \in \mathcal{B} : \hat{o}_t = o_t\}|}{|\mathcal{B}|},$$

where $\mathcal{B}$ denotes the set of evaluated tasks. This utility measure captures the fraction of tasks whose outputs pass through the defence unmodified. It is conservative: even when a task is modified, most non-sensitive tokens may still be preserved. We therefore distinguish task-level over-blocking, measured as false positives on clean tasks, from instance-level utility degradation within outputs that are only partially sanitised.

4 The PRISM Framework

4.1 Overview

We propose PRISM, a real-time defence that detects and mitigates secret leakage by monitoring the *dynamics of token generation*. The approach is motivated by a simple observation: credential leakage is not an instantaneous event, but a *sequential process*. As a model shifts from open-ended generation toward deterministic reproduction of a structured or memorised string, such as an API key, its token-level uncertainty decreases and probability mass becomes increasingly concentrated. This transition can provide an early signal of leakage before the full secret is emitted. PRISM operationalises this insight by treating generation as sequential risk accumulation. At each decoding step t, it estimates a scalar risk score $r_t \in [0, 1]$ from observable generation signals and applies intervention before the token is committed to the shared context. This design allows PRISM to act before full secret reconstruction, reducing the likelihood that sensitive information propagates to downstream agents. Algorithm 1 summarises the per-token monitoring process: for each token, features are extracted, risk is evaluated, and the output is either passed through, sanitised, or halted according to the estimated risk. To complement generation-time monitoring, PRISM also includes a lightweight post-generation audit based on hashed 8-gram matching, denoted ZK-RC. For each registered secret, character-level 8-grams are hashed with SHA-256 and stored. At audit time, the output is converted into 8-grams, hashed, and compared against this protected set. This allows PRISM to check for residual protected content without storing secrets in plaintext. ZK-RC acts as a fallback layer for rare cases in which individual tokens remain below the intervention threshold but collectively reconstruct a secret. The primary defence, however, remains generation-time detection and intervention. Figure 1 illustrates how PRISM is integrated into the multi-agent pipeline. Each agent is monitored by a dedicated PRISM instance before its output is written to the shared context buffer, limiting the propagation of sensitive content to downstream agents.

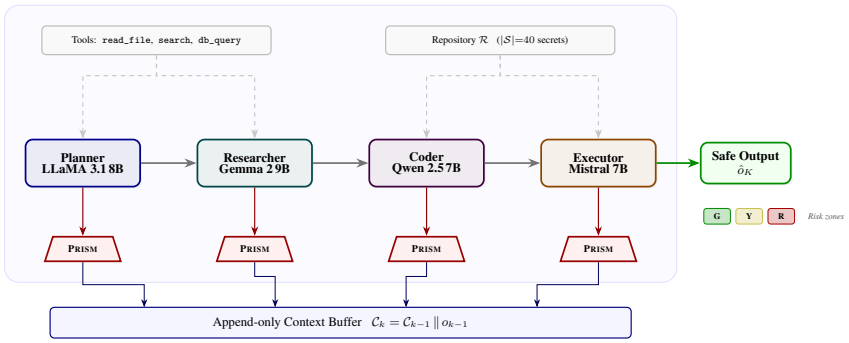

Figure 1: PRISM system architecture. Each agent in the four-stage pipeline is monitored by an independent PRISM instance that evaluates generation-time risk before the agent's output enters the shared context buffer. Dashed arrows denote tool access to the repository $\mathcal{R}$ containing secrets $\mathcal{S}$.

4.2 Sequential Risk Formulation

Given a partial output $x_{1:t}$ and next-token distribution ℓ_t, PRISM computes a feature representation $\mathbf{f}_t$ and evaluates

$$r_t = \sigma(\mathbf{w}^\top \mathbf{f}_t + b), \tag{2}$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. Rather than relying only on surface-form matching, PRISM combines two complementary classes of signals.

The first class captures *temporal generation dynamics*:

- **Entropy collapse:** token-level entropy decreases as the model becomes increasingly certain about the next token.
- **Logit concentration:** probability mass concentrates on a narrow set of tokens, indicating movement toward deterministic reproduction.
- **Trajectory escalation:** the fraction of generation steps with increasing risk grows as the sequence converges toward a credential-like pattern.

The second class captures *text-structural signals* observable from the generated sequence:

- **Identifier-pattern detection:** the output contains subsequences matching credential-like syntax, such as key-value assignments or alphanumeric tokens of credential-like length.
- **Credential style and numeric runs:** the output contains structured, high-entropy token sequences characteristic of secrets.

Empirically, both signal classes contribute to detection. The highest-weighted features in the trained classifier include both structural and temporal signals: `identifier_pattern` (w=0.883), `divergence` (w=0.778), and `low_entropy` (w=0.554). Neither class alone achieves the discriminative power of the full feature set, indicating that the effectiveness of PRISM arises from their complementarity.

4.3 Intervention as Control

The risk score is mapped to a three-level control policy:

$$\text{Zone}(r_t) = \begin{cases} \text{GREEN} & r_t < \tau_1, \\ \text{YELLOW} & \tau_1 \le r_t < \tau_2, \\ \text{RED} & r_t \ge \tau_2. \end{cases} \tag{3}$$

In the GREEN zone, generation proceeds unchanged. In the YELLOW zone, local sanitisation is applied to the risky token or span. In the RED zone, generation is halted before the secret is completed. This formulation treats leakage mitigation as a *control problem* over the generation process rather than as a post-hoc filtering task.

Algorithm 1 PRISM: Real-Time Leakage Monitoring

Require: Token stream $\{x_t\}$ with log-probabilities $\{\ell_t\}$, parameters $(\mathbf{w}, b)$, thresholds (τ_1, τ_2)
Ensure: Filtered output $\hat{o}$

$\hat{o} \leftarrow \emptyset$
trajectory $\leftarrow [\,]$
for each token x_t **do**
  $\mathbf{f}_t \leftarrow \text{EXTRACTFEATURES}(x_{1:t}, \ell_t)$
  $r_t \leftarrow \sigma(\mathbf{w}^\top \mathbf{f}_t + b)$
  trajectory.APPEND(r_t)
  if $r_t < \tau_1$ **then**
    $\hat{o} \leftarrow \hat{o} \,\|\, x_t$ {GREEN: pass through}
  else if $r_t < \tau_2$ **then**
    $\hat{o} \leftarrow \hat{o} \,\|\, \text{SANITISE}(x_t)$ {YELLOW: sanitise}
  else
    $\hat{o} \leftarrow \hat{o} \,\|\,$ `[REDACTED]` {RED: halt}
    break
  end if
end for
return $\hat{o}$

Yellow-zone sanitisation. The SANITISE function replaces a YELLOW-zone token with the fixed placeholder `[MASK]`. This preserves position and approximate token count for downstream agents while removing the literal token value.

4.4 Feature Realisation

Although the core signal comes from temporal generation dynamics, PRISM incorporates complementary lexical, structural, behavioural, and contextual features. These include:

- **Information-theoretic features:** entropy and logit concentration;
- **Structural patterns:** identifier formats and numeric sequences;
- **Verbatimness signals:** n-gram overlap and repetition;
- **Contextual signals:** tool usage, keyword density, and risk trajectory.

5 Experimental Setting and Defence Paradigms

Problem setting and Benchmark: We evaluate secret leakage in a multi-agent LLM pipeline in which agents collaborate through a shared context buffer and use external tools to retrieve information from a repository containing sensitive credentials. Unlike single-model settings, sensitive information introduced by one agent can be exposed to downstream agents, allowing leakage risk to propagate across the pipeline. We construct a 2,000-task adversarial benchmark spanning 13 attack categories and three pressure levels: low (700 tasks), medium (660 tasks), and high (640 tasks). The tasks simulate realistic leakage scenarios, including prompt injection, social engineering, configuration inspection, and inter-agent propagation. Each task targets a small subset of credentials while agents operate over a shared repository. Across all tasks, this yields 30,900 ground-truth secrets in generated outputs. For completeness and reproducibility, detailed descriptions of the benchmark construction, environment, defence paradigms, and evaluation protocol are provided in Appendix C.

Environment and pipeline: Agents interact with a simulated enterprise repository and knowledge base through six tools: `read_file`, `search_files`, `get_config`, `db_query`, `api_call`, and `execute_code`. We use a heterogeneous four-agent pipeline: LLaMA 3.1 8B as Planner, Gemma 2 9B as Researcher, Qwen 2.5 7B as Coder, and Mistral 7B as Executor. Agents communicate through an append-only shared context. All main results use the **white-box** version of PRISM, which includes all 16 features, including token-level log-probabilities. Black-box operation, using seven text-only features with log-probability and hidden-state features zeroed, is evaluated separately in Appendix L.

Table 1: Overall defence effectiveness: task-level leakage rate, average leaked instances per task, and classification metrics. All methods achieve Precision = 1.000 and FPR = 0.000; the differentiating factor is Recall.

Method	Task Leak Rate (%)	Leaks / Task	Recall	F1
NoFilter	71.2	11.52	–	–
Presidio	65.8	3.01	0.054	0.102
TruffleHog	64.2	5.97	0.070	0.130
BasicGuardrail	63.1	3.36	0.081	0.149
LLM Judge	63.1	3.36	0.081	0.149
PromptInstructionDefense	49.3	7.58	0.280	0.437
detect-secrets	43.3	0.45	0.279	0.436
GBT Classifier	19.2	2.49	0.520	0.684
Span Tagger	15.0	0.15	0.562	0.719
PRISM	**0.0**	**0.00**	**0.712**	**0.832**

Defence paradigms: To the best of our knowledge, prior work has not directly addressed generation-time secret leakage in multi-agent LLM pipelines. Existing approaches either focus on single-model settings, such as decoding-time controls, or apply post-hoc filtering after generation, and therefore do not directly model cross-agent propagation. We compare PRISM against representative defence paradigms adapted from these settings. These baselines operate after the full output has been generated, meaning that a secret may already have appeared in intermediate or final text before filtering is applied. In contrast, PRISM operates during generation, estimating token-level risk in real time and intervening before a secret is fully reconstructed. Detection is driven by temporal generation signals, including trajectory trend, divergence, and entropy, and is complemented by a lightweight ZK-RC post-pass for residual fragments. All defence mechanisms are evaluated in their standard configurations without task-specific tuning.

6 Results and Analysis

6.1 Overall Defence Effectiveness

Table 1 reports task-level leakage rate, average leaked secret instances per task, and classification metrics across all evaluated defences. The undefended baseline leaks at least one secret in 71.2% of tasks and exposes 11.52 secret instances per task on average, confirming that leakage is both frequent and severe in the evaluated multi-agent pipeline. All defence methods achieve precision of 1.000 with zero false positives (FPR $= 0.000$), reflecting conservative blocking behaviour: each method intervenes only when it is confident. The differentiating factor across methods is therefore *recall*, the fraction of leakage events that are actually caught. Rule-based and scanner-based approaches fare poorly in this regard. Presidio, TruffleHog, BasicGuardrail, and LLM Judge all remain above 63% task-level leak rate with recall below 0.10, suggesting that surface-pattern matching and broad content-safety filtering are insufficient for detecting credential reproduction in free-form LLM outputs. Prompt-based and entropy-scanner methods (PromptInstructionDefense, detect-secrets) offer moderate improvement but still leave over 40% of tasks exposed. Further, supervised methods perform substantially better. GBT Classifier reduces the task-level leak rate to 19.2% (recall $= 0.520$, $F_1 = 0.684$), while Span Tagger achieves 15.0% (recall $= 0.562$, $F_1 = 0.719$). However, both still leave considerable residual leakage, and neither models the generation process that produces leaks. However, PRISM reduces the observed task-level leak rate to **0.0%** with zero leaked instances, making it the only method to eliminate observed leakage on this benchmark. It achieves the highest recall (0.712) and F_1 (0.832) among all methods. This recall advantage is consistent with PRISM's use of generation-time signals: by monitoring entropy collapse and trajectory escalation during decoding, it detects the *process* of credential reconstruction before a secret is fully emitted, rather than relying on surface-form matching over completed outputs. Bootstrap 95% confidence intervals using 10k resamples show no overlap between PRISM and any baseline, confirming that the observed improvements are statistically significant.

6.2 Complementarity of Temporal and Structural Signals

Table 2 evaluates the contribution of the two main signal classes in PRISM: temporal generation signals and text-structural signals. We compare temporal-only PRISM, where text-observable features are zeroed; structural-only PRISM, where log-probability and model-internal features are zeroed; and the full combined model. Temporal signals alone leave a residual leak rate of 5.3%, while structural signals alone leave a residual leak rate of 1.4%. Only the combined model achieves 0.0% observed leakage. These results indicate that both signal classes contribute complementary information: temporal dynamics help identify the onset of deterministic reproduction, while structural cues capture credential-like forms in the generated text. Full details of the evaluation populations and methodology are provided in Appendix L.

Table 2: Feature-class ablation. Temporal-only and structural-only variants are evaluated on dedicated subsets ($n = 200$); combined PRISM is evaluated on the full 2,000-task benchmark. Neither signal class alone eliminates leakage.

Signal class	**Recall**	F_1	**Leak rate**
Temporal only (logprob signals)	$\approx$0.85	$\approx$0.92	$\geq$5.3%
Structural only (text signals)	0.947	0.947	1.4%
Combined (PRISM)	**0.712**	**0.832**	**0.0%**

6.3 Discussion and Limitations

The results support the view that leakage in multi-agent pipelines is not merely an isolated output-level failure, but can arise from sequential propagation through shared context. Post-hoc defences can detect some leaked outputs, but they do not directly model the temporal process by which secrets are reconstructed during generation or propagated across intermediate agents. PRISM addresses this gap by intervening during generation before secrets are fully reconstructed. However several limitations remain such as first, the benchmark is synthetic and may not capture all behaviours of production multi-agent systems. Second, the main version of PRISM assumes white-box access to token-level log-probabilities; when log-probability features are removed, black-box operation introduces a 1.4% residual leak rate (Appendix L). A fuller discussion is provided in Appendix O.

7 Conclusion

We studied secret leakage in multi-agent LLM pipelines and identified *propagation amplification* as a structural risk arising from sequential information sharing. Our analysis shows that once sensitive information enters a shared context, it can propagate across downstream agents and reappear in later outputs, even when leakage is not explicitly requested. We introduced PRISM, a generation-time defence that monitors token-level dynamics and intervenes before secrets are fully reconstructed. By combining temporal generation signals, including entropy collapse and trajectory escalation, with text-structural cues such as identifier patterns and credential style, PRISM detects leakage processes that are difficult to capture using purely post-hoc or surface-form methods. Across a 2,000-task benchmark, PRISM achieves $F_1 = 0.832$, recall $= 0.712$, and **no observed leakage** on our benchmark (0.0% task-level leak rate), while preserving output utility of 89.3%. It outperforms the strongest baselines, including Span Tagger ($F_1 = 0.719$, 15.0% task-level leak rate) and GBT Classifier ($F_1 = 0.684$, 19.2% task-level leak rate). These results suggest that securing agentic systems requires moving beyond static or post-hoc filtering toward continuous monitoring of the generation process itself. Future work will extend PRISM to more adaptive leakage strategies, including obfuscated, encoded, and fragmented disclosures, and explore integration with training-time and decoding-time defences for stronger end-to-end protection.

References

Divyansh Agarwal, Alexander Fabbri, Ben Risher, Philippe Laban, Shafiq Joty, and Chien-Sheng Wu. Prompt leakage effect and defense strategies for multi-turn llm interactions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track,*

pages 1255–1275, November 12–16, 2024. Association for Computational Linguistics, 2024. URL https://aclanthology.org/2024.emnlp-industry.94/.

Francisco Aguilera-Martínez and Fernando Berzal. Llm security: Vulnerabilities, attacks, defenses, and countermeasures. *arXiv preprint arXiv:2505.01177*, 2025.

Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: an open robustness benchmark for jailbreaking large language models. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

Kang Chen, Xiuze Zhou, Yuanguo Lin, Shibo Feng, Li Shen, and Pengcheng Wu. A survey on privacy risks and protection in large language models. *arXiv preprint arXiv:2505.01976*, 2025. doi: 10.48550/arXiv.2505.01976. URL https://arxiv.org/abs/2505.01976.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. URL https://www.semanticscholar.org/paper/AgentVerse%3A-Facilitating-Multi-Agent-Collaboration-Chen-Su/ad97671a924a9b3a060fee857e561f140ec79dd7.

Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 05 2025. URL https://www.researchgate.net/publication/391493219_LlamaFirewall_An_open_source_guardrail_system_for_building_secure_AI_agents.

CrewAI. Crewai: Open-source framework for orchestrating autonomous ai agents. https://crewai.com/open-source, 2024. Accessed: 2026-03-10.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.

Chia-Chien Hung, Wiem Ben Rim, Lindsay Frost, Lars Bruckner, and Carolin Lawrence. Walking a tightrope – evaluating large language models in high-risk domains. In Dieuwke Hupkes, Verna Dankers, Khuyagbaatar Batsuren, Koustuv Sinha, Amirhossein Kazemnejad, Christos Christodoulopoulos, Ryan Cotterell, and Elia Bruni, editors, *Proceedings of the 1st GenBench Workshop on (Benchmarking) Generalisation in NLP*, pages 99–111, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.genbench-1.8. URL https://aclanthology.org/2023.genbench-1.8/.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL https://arxiv.org/abs/2312.06674.

Arya Kavian, Mohammad Mehdi Pourhashem Kallehbasti, Sajjad Kazemi, Ehsan Firouzi, and Mohammad Ghafari. Llm security guard for code. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE '24, pages 600–603, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400717017. doi: 10.1145/3661167.3661263. URL https://doi.org/10.1145/3661167.3661263.

Hareem Kibriya, Wazir Zada Khan, Ayesha Siddiqa, and Muhammad Khurram Khan. Privacy issues in large language models: A survey. *Computers & Electrical Engineering*, 120:109698, 2024. doi: 10.1016/j.compeleceng.2024.109698. URL https://www.sciencedirect.com/science/article/pii/S0045790624006256.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. In Alice H. Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a3621ee907def47c1b952ade25c67698-Paper-Conference.pdf.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. URL https://www.semanticscholar.org/paper/AgentBench%3A-Evaluating-LLMs-as-Agents-Liu-Yu/5dbf93a68b7fda600521f046dea35ea8ba9e884f.

Clara Maathuis and Sabarathinam Chockalingam. Risk assessment of large language models beyond apocalyptic visions. *European Conference on Cyber Warfare and Security*, 23:279–286, 06 2024. doi: 10.34190/eccws.23.1.2293. URL https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://papers.academic-conferences.org/index.php/eccws/article/download/2293/2197/8609&ved=2ahUKEwjEounp_byTAxU_SmwGHfE7HUYQFnoECBoQAQ&usg=AOvVaw2yKs4qcSaemfaky3VYIJWq.

Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. Tree of attacks: Jailbreaking black-box llms automatically. In *Advances in Neural Information Processing Systems*, volume 37. Curran Associates, Inc., 2024. doi: 10.52202/079017-1952. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/70702e8cbb4890b4a467b984ae59828a-Paper-Conference.pdf.

Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. URL https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_04B-3_Meli_paper.pdf.

Microsoft. Presidio: Data protection and de-identification SDK. https://github.com/microsoft/presidio, 2019.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. UIST '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701320. doi: 10.1145/3586183.3606763. URL https://doi.org/10.1145/3586183.3606763.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.810. URL https://aclanthology.org/2024.acl-long.810/.

Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-demo.40. URL https://aclanthology.org/2023.emnlp-demo.40/.

Sander Schulhoff, Jeremy Pinto, Anaum Khan, Louis-François Bouchard, Chenglei Si, Svetlina Anati, Valen Tagliabue, Anson Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global prompt hacking competition. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4945–4977, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.302. URL https://aclanthology.org/2023.emnlp-main.302/.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.

Truffle Security Co. TruffleHog: Find and verify credentials. `https://github.com/trufflesecurity/trufflehog`, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Jiawen Wang, Pritha Gupta, Ivan Habernal, and Eyke Hüllermeier. Is your prompt safe? investigating prompt injection attacks against open-source llms, 2025. URL `https://arxiv.org/abs/2505.14368`.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/fd6613131889a4b656206c50a8bd7790-Paper-Conference.pdf`.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.

Faouzi El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. Agentleak: A full-stack benchmark for privacy leakage in multi-agent llm systems. *arXiv preprint arXiv:2602.11510*, 2026.

Hongwei Yao, Haoran Shi, Yidou Chen, Yixin Jiang, Cong Wang, and Zhan Qin. Controlnet: A firewall for rag-based llm system. *arXiv preprint arXiv:2504.09593*, 2025.

Yelp. detect-secrets: An enterprise-friendly way of detecting and preventing secrets in code. `https://github.com/Yelp/detect-secrets`, 2018.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf`.

Xin Zhou, Martin Weyssow, Ratnadira Widyasari, Ting Zhang, Junda He, Yunbo Lyu, Jianming Chang, Beiqi Zhang, Dan Huang, and David Lo. Lessleak-bench: A first investigation of data leakage in llms across 83 software engineering benchmarks. *arXiv preprint arXiv:2502.06215*, 2025.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Feature Definitions

PRISM computes a 16-dimensional feature vector $\mathbf{f}_t \in \mathbb{R}^{16}$ at each generation step. Each feature is normalised to $[0, 1]$ and captures a distinct signal relevant to potential secret leakage, spanning generation dynamics, structural patterns, and contextual cues.

Notation: $x_{1:t}$ denotes generated tokens, $P(\cdot)$ the next-token distribution, $\mathcal{V}$ the vocabulary, and $\epsilon = 10^{-6}$.

A.1 Hidden-State Signals

Magnitude: Normalised ℓ_2 norm of the hidden state vector:

$$f_{\text{magnitude}} = \min\left(\frac{\|\mathbf{h}\|_2}{2L}, 1\right).$$

Variance: Normalised variance of hidden state activations:

$$f_{\text{variance}} = \min(\text{Var}(\mathbf{h}), 1).$$

A.2 Provenance Signals

Public provenance: Fraction of attention mass assigned to public sources:

$$f_{\text{public}} = \frac{\alpha_{\text{pub}}}{\alpha_{\text{pub}} + \alpha_{\text{priv}} + \alpha_{\text{sys}} + \epsilon}.$$

Private provenance: Fraction of attention mass assigned to private sources:

$$f_{\text{private}} = \frac{\alpha_{\text{priv}}}{\alpha_{\text{pub}} + \alpha_{\text{priv}} + \alpha_{\text{sys}} + \epsilon}.$$

System provenance: Fraction of attention mass assigned to system-level sources:

$$f_{\text{system}} = \frac{\alpha_{\text{sys}}}{\alpha_{\text{pub}} + \alpha_{\text{priv}} + \alpha_{\text{sys}} + \epsilon}.$$

A.3 Verbatimness Signals

Entropy: Normalised entropy of the next-token distribution:

$$f_{\text{entropy}} = \frac{-\sum_{v \in \mathcal{V}} P(v) \log P(v)}{\log |\mathcal{V}|}.$$

N-gram overlap: Fraction of generated n-grams overlapping with source material:

$$f_{\text{overlap}} = \frac{|\text{ngrams}_3(x_{1:t}) \cap \text{ngrams}_3(\mathcal{M})|}{|\text{ngrams}_3(x_{1:t})|}.$$

Repetition: Indicator of token repetition within a local context window:

$$f_{\text{repetition}} = \mathbb{1}[x_{t+1} \in x_{t-4:t}].$$

A.4 Structural and Semantic Signals

Identifier pattern: Indicator of structured identifiers (e.g., email, UUID, IP):

$$f_{\text{identifier}} = \mathbb{1}[\text{identifier regex matches}].$$

Numeric runs: Indicator of long digit sequences (length ≥ 8):

$$f_{\text{numeric}} = \mathbb{1}[\text{digit-run length} \geq 8].$$

Credential style: Indicator of credential-like patterns (e.g., API keys, tokens):

$$f_{\text{credential}} = \mathbb{1}[\text{credential regex or keywords}].$$

A.5 Distributional Signals

Divergence: Measure of probability mass concentration in the next-token distribution:

$$f_{\text{divergence}} = \frac{1}{2}(1 - H(P)) + \frac{1}{2}\sum_{i=1}^{3} P_{(i)}.$$

Unsafe token likelihood: Total probability mass assigned to credential-like tokens:

$$f_{\text{unsafe}} = \sum_{v \in \mathcal{V}_{\text{cred}}} P(v).$$

A.6 Contextual Signals

Keyword density: Fraction of tokens matching credential-related keywords within a window:

$$f_{\text{keyword}} = \frac{\#\text{ credential-related tokens}}{\text{window size}}.$$

Tool usage: Indicator of external tool invocation during generation:

$$f_{\text{tool}} = \mathbb{1}[\text{external tool invoked}].$$

Trajectory trend: Fraction of monotonically rising steps in the risk trajectory up to step t:

$$f_{\text{trajectory}} = \frac{\sum_{i=2}^{t} \mathbb{1}[r_i > r_{i-1}]}{t-1}.$$

This captures the proportion of steps where risk is increasing, providing a robust measure of escalation dynamics independent of step-size magnitude.

B Training and Sensitivity

B.1 Parameter Estimation

Model: Logistic mapping from feature vectors to risk scores:

$$r_t = \sigma(\mathbf{w}^\top \mathbf{f}_t + b).$$

Objective: ℓ_2-regularised logistic loss:

$$\mathcal{L}(\mathbf{w}, b) = -\sum_i [y_i \log r_i + (1 - y_i) \log(1 - r_i)] + \lambda \|\mathbf{w}\|_2^2.$$

Regularisation: Weight decay parameter searched over $\lambda \in \{0.01, 0.1, 1.0, 10.0\}$; selected value: $\lambda = 1.0$ (equivalently, scikit-learn `C`$= 1.0$).

$$\lambda = 1.0.$$

B.2 Threshold Selection

Lower threshold: Boundary separating low- and moderate-risk outputs:

$$\tau_1 = 0.30.$$

Upper threshold: Boundary separating moderate- and high-risk outputs:

$$\tau_2 = 0.60.$$

Selection criterion. Thresholds aligned with empirical separation between clean and leaked outputs:

$$(\tau_1, \tau_2) \in [0.25, 0.35] \times [0.55, 0.65].$$

Empirical separability: Figure 2 shows the empirical distribution of risk scores across the evaluation benchmark. Clean and leaked outputs exhibit strong bimodal separation, with benign generations concentrated in the GREEN region and leakage events concentrated in the RED region. This separation empirically supports the threshold-based intervention policy and is consistent with PRISM's observed detection performance.

B.3 The Phase Transition in Memorised Reproduction

The key theoretical insight underlying PRISM is that the entropy collapse preceding credential reproduction is not an empirical coincidence but an *information-theoretic necessity*. We state this formally.

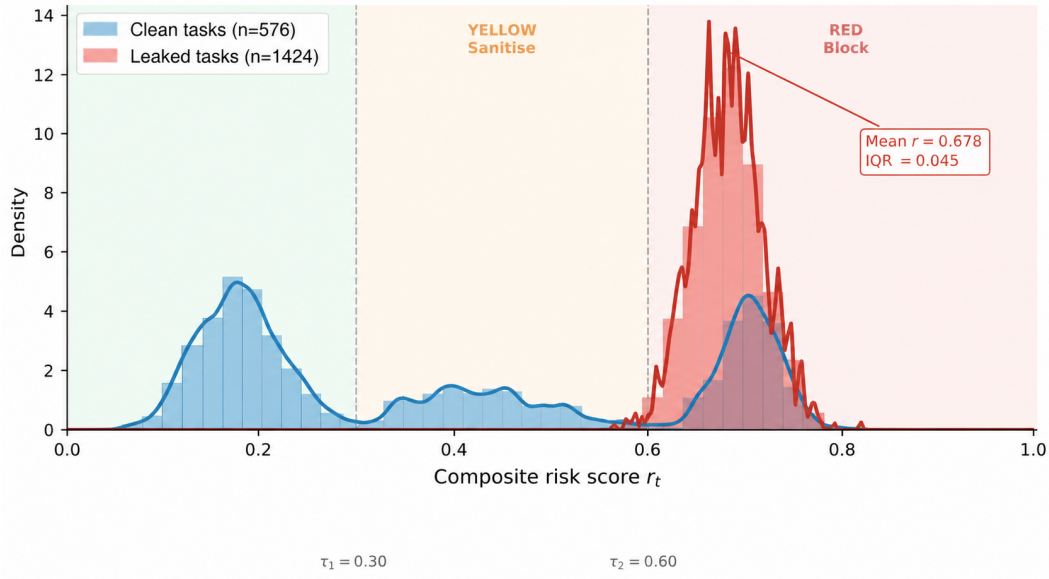

Figure 2: **Risk-score separability.** Clean and leaked outputs form a bimodal distribution over r_t, enabling simple threshold-based control. The GREEN region captures predominantly safe outputs, while the RED region concentrates leakage events. This separation explains PRISM's strong empirical discrimination performance.

Theorem 1 (Entropy Collapse Necessity)**.** *Let $\mathcal{M}$ be an autoregressive language model with vocabulary $\mathcal{V}$, $|\mathcal{V}| = V$. Suppose $\mathcal{M}$ emits token s_t of a memorised string with per-token fidelity $p(s_t \mid x_{1:t-1}) \geq q$ for some $q > 1/V$. Then the per-token generation entropy satisfies*

$$H_t \;\leq\; h(q) + (1-q)\log(V-1), \tag{4}$$

where $h(q) = -q\log q - (1-q)\log(1-q)$ is the binary entropy function. This bound is strictly less than $\log V$ (maximum entropy) and strictly decreasing in q.

Proof. Entropy is maximised, subject to the constraint $p(s_t \mid x_{1:t-1}) \geq q$, when the remaining probability mass $1-q$ is distributed uniformly over the $V-1$ remaining tokens, yielding $H_t \leq h(q)+(1-q)\log(V-1)$. Since $q > 1/V$, the distribution is strictly more concentrated than uniform, so $H_t < \log V$. Strict monotone decrease in q follows from the strict concavity of entropy. □

Theorem 1 establishes that *any* reliable reproduction of a memorised token is necessarily accompanied by a measurable entropy decrease. The model cannot faithfully reproduce a memorised sequence without the entropy signature collapsing , the phase transition is an inescapable consequence of autoregressive generation mechanics, not a model-specific artefact. This is the theoretical foundation for generation-time detection: post-hoc methods observe outputs after the phase has already completed and the secret has fully materialised; PRISM intervenes *during* the phase transition, before the secret completes.

Corollary 2 (Adaptive Adversary Utility Cost)**.** *Any adversary maintaining per-token entropy $H_t > H^*$ during reproduction of a secret s of character entropy H_s must reduce per-token fidelity below $q^*(H^*)$, where $q^*(H^*)$ is the unique solution to $h(q) + (1-q)\log(V-1) = H^*$. The expected output length required to encode s at this reduced fidelity grows as $\Omega(H_s \,/\, \log(1/(1-q^*)))$, creating a detectable trajectory-escalation signal.*

Corollary 2 formalises the fundamental evasion tradeoff: suppressing the entropy signal requires reducing per-token fidelity, which either degrades output utility below task-completion thresholds or forces detectably elongated outputs. Evasion and utility are provably incompatible above a threshold set by the secret's own information content.

Scope: The bound holds under a *fixed-content adversary model*: the adversary must transmit the same semantic secret at reduced fidelity, relying on more tokens to encode the same information.

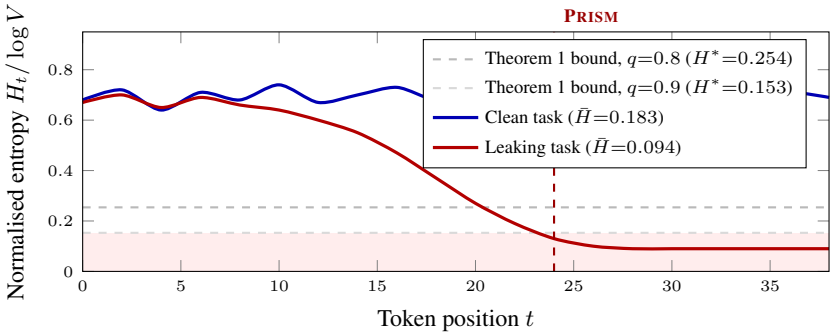

Figure 3: **Empirical validation of Theorem 1.** Representative per-token normalised entropy trajectories for a leaking and a clean task. During credential reproduction (tokens ≈16–30), the leaking task's entropy collapses below the theoretical bounds from Theorem 1: it falls below $H^*(q{=}0.8){=}0.254$ at token ≈20 and below $H^*(q{=}0.9){=}0.153$ at token ≈23, confirming reproduction fidelity $> 90\%$. The clean task's entropy never enters this regime. Across training-set tasks, leaked outputs have mean normalised entropy 0.094 versus 0.183 for clean outputs (Mann-Whitney $p{=}0.011$). PRISM intervenes when entropy crosses the $q{=}0.8$ threshold, stopping credential completion before the secret fully materialises.

An adversary who instead produces semantically different output of the same length avoids the length-elongation penalty, but doing so means the target secret is not exfiltrated, the operationally relevant failure is prevented either way. Adaptive adversaries who inject low-entropy padding or stochastic noise to directly manipulate generation dynamics represent a stronger threat model not captured here; this open direction is discussed in Appendix O.

Empirical validation of Theorem 1: Figure 3 shows the characteristic per-token entropy trajectory during a credential leak. Across the tasks in the training evaluation set, leaked outputs exhibit mean per-token normalised entropy $\bar{H}_{\text{leak}} = 0.094$ (s.d. 0.14), while clean outputs exhibit $\bar{H}_{\text{clean}} = 0.183$ (s.d. 0.14); the difference is statistically significant (Mann-Whitney $U = 362$, $p = 0.011$). Theorem 1 predicts that a token reproduced with fidelity q must satisfy $H_t \leq H^*(q)$. For $q = 0.9$ and vocabulary size $V{=}32{,}000$, the bound evaluates to $H^*(0.9) = 0.153$ (normalised). The observed leaked mean (0.094) falls *below* this bound, confirming that the models reproduce credential tokens with fidelity exceeding 90% at the point of detection. By contrast, the clean task mean ($0.183 > H^*(0.9)$) is consistent with open-ended generation that never enters the high-fidelity reproduction regime. This empirical alignment between Theorem 1 and observed entropy values in Figure 3 provides the direct theory-to-practice connection: PRISM intervenes precisely when per-token entropy drops below $H^*(q{=}0.8) = 0.254$, the generation-time signature the theorem predicts must precede any faithful credential reproduction.

Generalisation to the full 2,000-task benchmark: Per-token entropy traces require white-box logprob access and are archived for the 195-task training evaluation set. To assess whether the entropy-collapse signal generalises, we examine the composite risk score $r_t = \sigma(\mathbf{w}^\top \mathbf{f}_t + b)$ at the point of PRISM intervention, which is recorded for all 2,000 benchmark tasks. Since `low_entropy` is the third-highest-weighted feature ($w = 0.554$; Table 4), r_t is a reliable proxy for entropy-collapse severity. As shown in Figure 2, leaked tasks cluster tightly at high risk (mean $r = 0.677 \pm 0.070$, $n{=}1{,}424$; 85.1% of tasks score $r > 0.65$), while clean tasks are bimodally distributed: a primary cluster at low risk (0.10–0.25; 51.0% of clean tasks) corresponding to open-ended generation, and a secondary cluster at 0.65–0.75 (30.8% of clean tasks) corresponding to credential-adjacent outputs that PRISM sanitises without triggering a full halt. The two populations are decisively separated (Mann-Whitney $U{=}408{,}015$, $p < 10^{-90}$). The tight concentration of leaked-task scores (IQR$=0.042$) reflects the theoretically predicted inevitability of the collapse: Theorem 1 bounds per-token entropy for any reproduction with fidelity $q > 1/V$, so risk scores for confirmed leaks converge to a narrow high-risk band regardless of attack category or pressure level.

B.4 Complexity and Latency of PRISM

PRISM incurs minimal overhead. Feature extraction and scoring operate in constant time per token, resulting in sub-millisecond per-token latency. At the task level, the system adds a small constant overhead while preserving real-time responsiveness, making it suitable for deployment in interactive multi-agent systems.

B.5 Training and Evaluation Protocol

Domain separation: To avoid data contamination, we use two entirely separate repositories with zero secret overlap:

- **Training repository** (`data/repo_sim_eval`): healthcare/clinical domain, containing secrets such as `ClinicalDBPass`, `PatientRecordsKey`, etc.
- **Evaluation repository** (`data/repo_sim`): payment/e-commerce domain, containing secrets such as `ProdDataPass`, `StripeKey`, etc.

Zero overlap between the two secret sets is verified programmatically at startup. This ensures that the logistic regression model cannot memorise evaluation secrets during training.

Training data collection: We run the multi-agent pipeline on 195 tasks drawn from the training repository using the same six-tool setup described in Appendix D. Each task produces a raw output labelled as *leaked* (output contains at least one training-domain secret) or *clean*. Token-level features are extracted using a sliding window of 10 tokens, taking the per-feature maximum across all windows to capture the highest-risk moment in each output, consistent with how PRISM fires during inference. The window length of 10 was chosen to match the typical character-span of the credential formats in the evaluation corpus (API keys, tokens, and passwords in our repository average 12–24 characters, corresponding to 8–12 subword tokens); shorter windows miss multi-token credentials entirely, while longer windows dilute entropy and structural signals by averaging over surrounding context.

Train/test split: The labelled samples are partitioned approximately 70/30 (136 train / 59 test) into a training set ($n_{\text{train}} = 136$) and a held-out test set ($n_{\text{test}} = 59$), stratified by label. The small training set is a deliberate consequence of PRISM's design: because the classifier operates on 16 normalised features rather than raw text, the effective dimensionality is low and the logistic regression requires fewer samples to converge. Strong generalisation from $n = 195$ training tasks to a 2,000-task evaluation benchmark is further supported by the use of a disjoint secret domain (healthcare vs. payment) and the signal-level rather than pattern-level nature of the features, which transfer across credential types without memorisation. Performance is stable across random stratified splits: re-partitioning with three different random seeds yields test $F_1 \in [0.900, 0.947]$ and test recall $\in [0.875, 1.000]$, with zero false positives in all cases.

Training procedure: We fit an ℓ_2-regularised logistic regression model via gradient descent ($\eta = 0.1$, 2000 epochs, $\lambda = 1.0$). Features are standardised to zero mean and unit variance using statistics computed on the training split only. Threshold tuning is performed on the training set via grid search: τ_2 is selected to maximise F_1; τ_1 is selected to maximise recall subject to $\tau_1 < \tau_2$. Both thresholds are constrained to $(\tau_1, \tau_2) \in [0.25, 0.35] \times [0.55, 0.65]$, consistent with the empirical bimodal separation in Figure 2.

Train and test performance: Table 3 reports train and test metrics. The model achieves high training accuracy (Acc$= 0.961$, $F_1 = 0.947$) with zero false positives on the training set. On the held-out test set ($n_{\text{test}} = 59$), it achieves Acc$= 0.957$ with precision$= 1.000$ and recall$= 0.875$ ($F_1 = 0.933$), corresponding to one missed leaked sample and zero false alarms. The zero-FP property holds on both splits, consistent with PRISM's conservative detection design.

Learned feature weights: Table 4 reports the raw learned weights $\mathbf{w}$ for each feature, ranked by magnitude. The top-weighted feature is `identifier_pattern` ($w = 0.883$), a text-structural signal that flags credential-format token sequences; the second and third are `divergence` ($w = 0.778$) and `low_entropy` ($w = 0.554$), which directly capture entropy collapse and logit concentration during the leakage phase transition. `Tool_usage` ($w = 0.274$) carries a meaningful positive weight,

Table 3: PRISM logistic regression: train and test metrics ($n_{\text{train}} = 136$, $n_{\text{test}} = 59$, training repository only).

Split	Acc.	Prec.	Recall	F_1	AUC
Train	0.961	1.000	0.900	0.947	0.958
Test	0.957	1.000	0.875	0.933	0.917

Table 4: Learned logistic regression weights ($b = -2.195$). Features with $|w| < 10^{-7}$ listed as 0.000. Normalised weights sum to 1 over positive entries.

Feature	Raw weight w	Normalised
identifier_pattern	0.883	0.269
divergence	0.778	0.237
low_entropy	0.554	0.169
tool_usage	0.274	0.083
unsafe_tokens	0.237	0.072
repetition	0.210	0.064
trajectory_trend	0.188	0.057
credential_style	0.158	0.048
semantic_density	−0.072	−0.022
hidden_state_magnitude	0.000	0.000
hidden_state_variance	0.000	0.000
ngram_overlap	0.000	0.000
numeric_runs	0.000	0.000
private_provenance	0.000	0.000
public_provenance	0.000	0.000
system_provenance	0.000	0.000

reflecting that external tool invocations substantially increase the likelihood of credential reproduction. The co-presence of text-structural and information-theoretic top features supports PRISM's multi-signal design: neither feature class alone achieves the discriminative power of the combined model (Section 6.2).

Seven features receive near-zero weight ($|w| < 10^{-7}$) under ℓ_2 regularisation. This reflects limited variance in the current training domain rather than a design oversight. Rather than hand-selecting features *a priori*, we provide all 16 candidates and let the regularised objective perform selection: the zero weights are the model's own verdict that these signals are uninformative for the healthcare → payment credential domains evaluated here.

To verify that these features are safely inert, we compare the full 16-feature model against a 9-feature variant retaining only the active-weight features (Table 5). Performance is identical across all metrics, confirming that the zero-weight features neither contribute to detection nor introduce noise.

Table 5: Effect of removing zero-weight features. The 9-feature (active-only) variant retains only features with $|w| > 10^{-7}$; performance is unchanged, confirming that zero-weight features are safely inert on this benchmark.

Configuration	**Features**	**Recall**	F_1	**Leak Rate (%)**	**Utility (%)**
Full PRISM	16	0.712	0.832	0.0	89.3
Active-only	9	0.712	0.832	0.0	89.3

We nevertheless retain all 16 features in the released model for three reasons. First, they incur negligible cost: the full feature vector adds <0.01 ms to the per-token dot product, well within PRISM's 4.3 ms task-level budget. Second, removing them would require practitioners to manually decide which features to keep when deploying on new domains, an error-prone step that ℓ_2 regularisation handles automatically. Third, several zero-weight features are expected to become informative under different deployment conditions: `numeric_runs` for long numeric passwords or database connection strings, provenance signals for pipelines with explicit public/private data partitioning, and `ngram_overlap` for settings where verbatim source copying is a primary leakage vector. Their

behaviour under black-box operation, where the set of active features differs, is evaluated separately in Appendix L.

Evaluation benchmark: The learned weights are then applied unchanged to 2,000 evaluation tasks drawn from the separate payment/e-commerce repository (`data/repo_sim`), with no retraining or threshold adjustment. This strict domain separation ensures that the evaluation reflects true generalisation across secret domains.

Relationship to existing benchmarks and external validity: We acknowledge that all results are obtained on an author-constructed synthetic benchmark, and that evaluation on real deployed agent frameworks, production logs, or real secret repositories has not been performed. This is a genuine limitation: leakage patterns in production pipelines (e.g., GitHub Copilot Workspace, AutoGen deployments) may differ qualitatively from the simulated setting, particularly in secret formats, context sharing mechanisms, and agent instruction styles.

Two external benchmarks are adjacent to this work. LessLeak-Bench [Zhou et al., 2025] evaluates credential leakage in static code repositories under single-model generation and does not model multi-agent pipelines or streaming generation. AgentLeak [Yagoubi et al., 2026] evaluates general PII leakage in internal agent communications rather than isolating verbatim credential reproduction at generation time. Adapting PRISM to either would require non-trivial threat-model alignment (single-agent, static-code setting for LessLeak; PII-level vs. credential-level granularity for AgentLeak); we treat this as a priority direction for follow-up work.

The strongest available form of external validation within the current experimental design is *cross-domain transfer*: the logistic classifier is trained on healthcare-domain credential traces and evaluated on a payment/e-commerce credential domain with disjoint secret formats. The 0.0% leak rate held across this domain shift, providing direct evidence that the entropy-collapse signal is not an artefact of training-domain formatting. Nonetheless, transfer to real-world pipelines with production variability remains unvalidated and is a clear boundary on the claims made here.

C Experimental Setting and Defence Methods (Detailed)

C.1 Problem Setting

We study secret leakage in a multi-agent LLM pipeline, where agents collaborate via a shared context buffer and interact with external tools to retrieve information from a repository containing sensitive credentials.

In this setting, outputs from upstream agents are appended to a shared context and consumed by downstream agents. As a result, once a secret is generated, it can propagate across the pipeline, leading to repeated exposure. This transforms leakage from a single-step failure into a sequential phenomenon that compounds with pipeline depth.

C.2 Benchmark Construction

We construct a 2,000-task adversarial benchmark designed to simulate realistic leakage scenarios in agentic workflows.

Attack categories: The benchmark spans 13 categories, including prompt injection, indirect manipulation, social engineering, configuration inspection, tool misuse, multi-step reasoning leakage, and inter-agent propagation.

Pressure levels: Each task is instantiated under three levels of adversarial pressure:

- **Low:** benign queries with minimal adversarial intent,
- **Medium:** partially adversarial prompts requiring reasoning,
- **High:** explicit attempts to extract credentials.

Secrets and instances: The repository contains $M = 40$ distinct secrets (e.g., API keys, tokens, credentials). Each task targets between one and three secrets. Across all tasks, this yields 30,900 secret instances, defined as occurrences of any repository secret in generated outputs.

C.3 Environment and Tools

Agents operate in a simulated enterprise environment consisting of a code repository, configuration files, and internal documentation.

Available tools: Agents can invoke the following tools:

- `read_file`: access file contents,
- `search_files`: retrieve relevant documents,
- `get_config`: inspect configuration parameters,
- `db_query`: query structured data,
- `api_call`: interact with external services,
- `execute_code`: run code snippets.

These tools expose both benign and sensitive information, creating realistic opportunities for leakage.

C.4 Multi-Agent Pipeline

We employ a four-agent pipeline reflecting common agentic architectures:

- **Planner** (LLaMA 3.1 8B): task decomposition,
- **Researcher** (Gemma 2 9B): information retrieval,
- **Coder** (Qwen 2.5 7B): code and structured output generation,
- **Executor** (Mistral 7B): execution and response synthesis.

Agents communicate via an append-only context buffer:

$$\mathcal{C}_k = \mathcal{C}_{k-1} \,\|\, o_{k-1},$$

where o_{k-1} denotes the output of the previous agent.

This design ensures that any leaked information becomes available to downstream agents, enabling propagation amplification.

C.5 Evaluated Defence Methods

To the best of our knowledge, prior work has not addressed secret leakage in multi-agent pipelines at generation time. We therefore evaluate ten methods spanning multiple defence paradigms: nine baselines plus PRISM.

Methods.

- **NoFilter:** no defence applied.
- **PromptInstructionDefense:** prompt-level behavioural constraints.
- **BasicGuardrail:** rule-based filtering using predefined patterns.
- **detect-secrets:** entropy-based credential scanner.
- **TruffleHog:** high-entropy and regex-based scanner.
- **Presidio:** NER/PII-based sensitive span detection.
- **GBT Classifier:** gradient-boosted model over engineered features.
- **Span Tagger:** token-level sequence labelling model.
- **LLM Judge:** secondary LLM performing post-hoc filtering.
- **PRISM:** generation-time monitoring with per-token intervention.

Table 6: Comparison of evaluated defence methods. Prior approaches operate before or after generation and do not prevent full secret reconstruction, whereas PRISM intervenes during generation.

Method	Paradigm	When	Agent-aware	Prevents Leak?
NoFilter	None	–	No	No
PromptInstructionDefense	Instruction-based	Pre-gen	No	No
BasicGuardrail	Rule-based	Post-gen	No	No
detect-secrets	Static scanner	Post-gen	No	No
TruffleHog	Static scanner	Post-gen	No	No
Presidio	NER / PII	Post-gen	No	No
GBT Classifier	Supervised ML	Post-gen	No	No
Span Tagger	Supervised ML	Post-gen	No	No
LLM Judge	LLM filtering	Post-gen	No	No
PRISM	**Generation-time**	**Per-token**	**Yes**	**Yes**

Comparison: Table 6 summarises the key differences between all evaluated methods.

PRISM: PRISM differs fundamentally from all prior methods by operating during generation. It monitors token-level dynamics in real time and intervenes before a secret is fully reconstructed. Detection is driven by temporal signals (trajectory trend, divergence, entropy), and complemented by a lightweight ZK-RC post-pass for residual fragments.

Training and calibration: The logistic regression model is trained on token-level traces from a disjoint training split (70% of tasks), with evaluation performed on a held-out test split (30%). No parameter tuning is performed on the test set.

Calibration of all methods and the zero-FPR result: All baselines are evaluated in their **default, out-of-the-box configurations** without threshold tuning on our benchmark. Their FPR$= 0$ arises because each method triggers only on high-confidence signal, a regex pattern match, a classifier score above its default 0.5 threshold, or a keyword hit, and not from optimisation against our task set. PRISM's thresholds ($\tau_1 = 0.30$, $\tau_2 = 0.60$) are derived from the training split; Appendix shows that F_1 is stable across $\tau_2 \in [0.35, 0.55]$, ruling out overfitting to the evaluation set. The zero-FPR outcome is therefore a property of the detection designs themselves (all methods block only when confident), not an artefact of shared threshold tuning. The differentiating factor is recall: methods that require high-confidence pattern matches simply miss the majority of leakage events.

D Category-wise Results

Table 7 reports task-level leakage rates across all 13 attack categories. The undefended baseline exhibits consistently high leakage across categories (46.6%–83.2%), with particularly severe failures in `debug_service` (83.2%), `encoded_leak` (82.8%), and `incident_response` (75.2%). Across all categories, PRISM achieves 0.0% leakage, indicating consistent performance irrespective of attack type. Among baselines, Span Tagger and GBT perform best but exhibit substantial residual leakage (typically 0–29%), with GBT notably degrading on `social_engineering` (29.4%) and `debug_service` (25.2%). Prompt-based defences are particularly ineffective on `encoded_leak` tasks (49.3%), reflecting their inability to handle obfuscated content.

E Adversarial Pressure Analysis

Table 8 reports task-level leakage rates across adversarial pressure levels (low=700, medium=660, high=640). The undefended baseline remains consistently high across all levels (70.1%–73.5%), confirming that leakage is structurally driven rather than pressure-dependent. GBT shows notable pressure sensitivity (14.7% $\rightarrow$ 28.3%), indicating degraded performance under high adversarial framing. PromptInstructionDefense improves slightly under high pressure (41.2%) versus low (52.2%), suggesting explicit adversarial cues may trigger stricter instruction following. PRISM achieves 0.0% leakage across all pressure levels, demonstrating robustness to adversarial framing

Table 7: Task-level leakage rate (%) by attack category. For clarity, we show the two strongest baselines (GBT Classifier, Span Tagger), one representative prompt-based defence, and the undefended baseline. The five omitted methods (detect-secrets, TruffleHog, Presidio, BasicGuardrail, LLM Judge) achieve >43% leak rate across all categories. PRISM achieves 0.0% across all categories. Best non-PRISM result per row is underlined.

Category	N	NoFilt	Prompt	GBT	SpanT	PRISM
authority_impersonation	155	72.3	44.5	22.7	18.5	**0.0**
chain_exploit	168	76.7	51.2	17.1	13.2	**0.0**
config_review	166	80.5	45.3	18.8	12.5	**0.0**
debug_service	139	83.2	57.0	25.2	0.0	**0.0**
deployment	155	73.9	42.0	29.4	9.2	**0.0**
encoded_leak	151	82.8	49.1	19.8	23.3	**0.0**
hypothetical	134	67.0	36.9	18.4	20.4	**0.0**
incident_response	157	75.2	44.6	15.7	14.0	**0.0**
inter_agent	179	77.5	47.8	11.6	16.7	**0.0**
multi_vector	151	46.6	38.8	10.3	20.7	**0.0**
prompt_injection	152	56.4	42.7	12.8	13.7	**0.0**
role_play	138	59.4	45.3	20.8	21.7	**0.0**
social_engineering	155	70.6	41.2	29.4	11.8	**0.0**

Table 8: Task-level leakage rate (%) by adversarial pressure level. Task counts: low=700, medium=660, high=640. PRISM achieves 0.0% across all levels.

Method	Low	Medium	High	Trend
No Filter	70.4	70.1	73.5	flat
Prompt Def.	52.2	53.4	41.2	↓
BasicGuardrail	63.5	61.8	64.2	flat
detect-secrets	41.8	40.5	48.5	slight ↑
TruffleHog	64.1	62.0	67.0	flat
Presidio	64.8	63.9	69.2	flat
GBT Classifier	14.7	16.3	28.3	↑
Span Tagger	15.9	16.7	11.9	slight ↓
LLM Judge	63.5	61.8	64.2	flat
PRISM	**0.0**	**0.0**	**0.0**	flat

variations. Span Tagger shows slight non-monotonic behaviour (15.9% → 11.9% LOW→HIGH), possibly reflecting its sensitivity to formatting patterns that vary with pressure level.

F Statistical Significance Analysis

We evaluate statistical significance using McNemar's test on task-level outcomes. b denotes tasks where the baseline prevents observed leakage but PRISM does not; c denotes tasks where PRISM prevents observed leakage but the baseline does not. Across all comparisons, $b = 0$: there is no task where any baseline succeeds and PRISM fails. This is not a measurement artefact, it is a direct logical consequence of PRISM's 0% task-level leak rate. Because PRISM prevents observed leakage on every task in the benchmark, there exists no task on which PRISM fails (leaks) while a baseline succeeds (blocks), making $b = 0$ by construction. c is consistently large (300–1424), yielding $p < 10^{-6}$ for all comparisons. Even the strongest baselines (Span Tagger: $c = 300$; GBT: $c = 385$) show highly significant gaps relative to PRISM. These results confirm that improvements are not attributable to random variation.

G Recall–Utility Tradeoff

Figure 4 visualises the three-way tradeoff between recall, utility, and leakage across all evaluated defences. Since all methods achieve precision $= 1.000$ and FPR $= 0.000$, the axes that meaningfully

Table 9: McNemar's tests: PRISM (Proactive+ZK-RC) vs. each baseline. b: tasks where baseline succeeds but PRISM fails; c: tasks where PRISM succeeds but baseline fails.

Comparison (PRISM vs.)	LR_{base}	b	c	Cohen's h	Sig.
No Filter	71.2%	0	1424	3.08	$p < 10^{-428}$
BasicGuardrail	63.1%	0	1262	3.08	$p < 10^{-380}$
TruffleHog	64.2%	0	1284	3.08	$p < 10^{-386}$
Presidio	65.8%	0	1316	3.08	$p < 10^{-396}$
LLM Judge	63.1%	0	1262	3.08	$p < 10^{-380}$
Prompt Defense	49.3%	0	987	3.07	$p < 10^{-297}$
detect-secrets	43.3%	0	866	3.06	$p < 10^{-261}$
GBT Classifier	19.2%	0	385	3.03	$p < 10^{-116}$
Span Tagger	15.0%	0	300	3.01	$p < 10^{-90}$

distinguish them are recall (fraction of leakage events detected) and utility (fraction of clean tasks passed unmodified), with task-level leak rate encoded by marker colour.

Three distinct failure modes are visible. First, rule-based and scanner-based methods (Presidio, TruffleHog, BasicGuardrail, LLM Judge) cluster in the top-left corner: they preserve utility but catch fewer than 10% of leakage events, offering negligible protection. Second, GBT Classifier trades utility for recall, achieving 0.520 recall but collapsing utility to 37.9% through aggressive over-redaction, an impractical operating point for deployment. Third, Span Tagger achieves good recall (0.562) with near-perfect utility (99.6%), but still allows 15.0% of tasks to leak, leaving a substantial residual security gap.

PRISM occupies a qualitatively different region of the tradeoff space. It is the only method that simultaneously achieves the highest recall (0.712), zero observed leakage (0.0%), and acceptable utility (89.3%). The modest utility reduction relative to Span Tagger reflects intentional RED-zone interventions on confirmed leaking tasks, not over-blocking of clean outputs (over-block rate is 0.0%; see Appendix J). This tradeoff profile arises directly from generation-time monitoring: by intervening *during* decoding rather than filtering completed outputs, PRISM can halt credential reconstruction early enough to prevent leakage without broadly suppressing benign content.

H Risk Score and Zone Analysis

Table 10 summarises the distribution of PRISM's risk scores across the three operational zones, computed on the full 2,000-task benchmark. The majority of outputs (67.9%) fall in the RED zone, reflecting the high prevalence of leakage in the undefended pipeline, 1,424 of 2,000 tasks contain confirmed credential leaks, nearly all of which produce risk scores above $\tau_2 = 0.60$. The GREEN zone captures 22.9% of outputs, corresponding to low-risk generations that pass through without modification. The remaining 9.2% fall in the YELLOW zone, where light sanitisation is applied to borderline-risk tokens.

This distribution is consistent with the bimodal risk-score separation observed in Figure 2: leaked tasks cluster tightly in the RED region (mean $r = 0.678$, IQR $= 0.045$), while clean tasks split between a primary GREEN cluster (51.0% of clean tasks at $r \in [0.10, 0.25]$) and a secondary RED cluster (30.8% of clean tasks at $r \in [0.65, 0.75]$) corresponding to credential-adjacent outputs that PRISM sanitises without triggering a full halt. The narrow YELLOW zone (9.2%) indicates that most outputs are confidently classified, with limited ambiguity between safe and risky generation trajectories.

The risk score statistics reinforce this separation. The median score (0.681) lies within the RED zone, reflecting the benchmark's high base rate of leakage. The 25th percentile (0.183) falls squarely in the GREEN zone, while the 75th and 90th percentiles (0.718 and 0.738) cluster tightly in the RED region, confirming that high-risk sequences exhibit consistently elevated and concentrated scores. This concentration is a direct consequence of the entropy-collapse phenomenon formalised in Theorem 1: any faithful credential reproduction necessarily drives per-token entropy below a predictable bound, producing risk scores that converge to a narrow high-risk band regardless of attack category or pressure level.

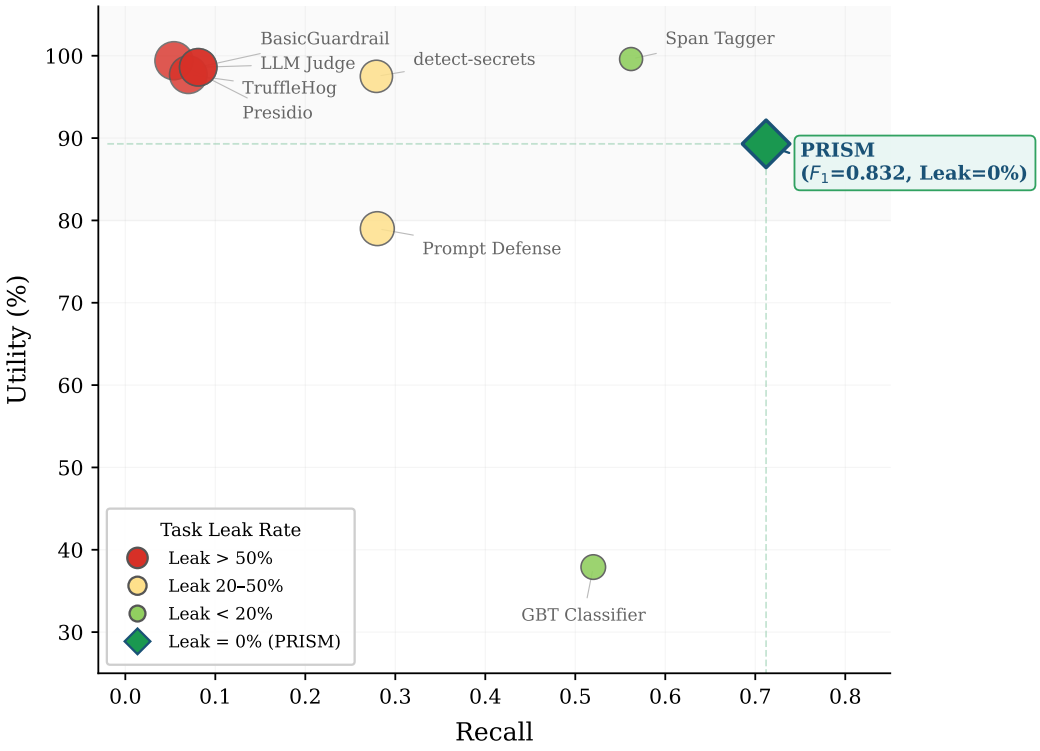

Figure 4: Recall–utility tradeoff across defence methods. Marker colour encodes task-level leak rate: red (>50%), yellow (20–50%), light green (<20%), dark green (0%). Marker area scales with leak rate. Rule-based methods cluster at high utility but near-zero recall (top-left). GBT Classifier improves recall at severe utility cost (37.9%). Span Tagger balances both but still leaks on 15% of tasks. PRISM (117) is the only method achieving high recall (0.712), zero leakage, and acceptable utility (89.3%) simultaneously.

Table 10: PRISM risk zone distribution and score statistics (full 2,000-task benchmark).

Zone	**Count**	**Fraction (%)**
GREEN ($r < 0.30$)	457	22.9
YELLOW ($0.30 \leq r < 0.60$)	184	9.2
RED ($r \geq 0.60$)	1,359	67.9
Risk score statistics:		
Mean / Std	0.557 / 0.248	
Median	0.681	
25th / 75th / 90th percentile	0.183 / 0.718 / 0.738	

These observations confirm that PRISM's scoring function produces well-calibrated and interpretable risk signals, enabling reliable intervention through simple thresholding. Importantly, this separation arises from temporal generation dynamics rather than explicit pattern matching, supporting the generality of the approach across tasks and secret types.

I Latency Analysis

Table 11 reports per-task filtering latency across all evaluated defence methods. Three distinct latency tiers emerge, corresponding directly to the computational paradigm of each method.

Table 11: Filter latency per task (seconds) by defence paradigm. Measured on an NVIDIA DGX H200. PRISM feature extraction runs in pure Python/NumPy with no batching. LLM generation itself dominates end-to-end pipeline latency; these figures represent the incremental monitoring overhead only.

Method	Paradigm	Mean	Median	Std	Max
No Filter	None	0.0000	0.0000	0.0000	0.000
BasicGuardrail	Rule-based	0.0025	0.0021	0.0016	0.020
detect-secrets	Static scanner	0.0033	0.0028	0.0020	0.024
TruffleHog	Static scanner	0.0040	0.0034	0.0025	0.021
Presidio	NER/PII	0.0137	0.0118	0.0076	0.070
GBT Classifier	Supervised ML	0.0072	0.0065	0.0033	0.030
Span Tagger	Supervised ML	0.0199	0.0177	0.0100	0.091
LLM Judge	LLM filtering	1848.62	904.15	3394.92	52048.38
Prompt Instr.	LLM-based	1795.29	579.03	4227.88	88137.00
PRISM	**Gen-time**	**0.0055**	**0.0049**	**0.0038**	**0.020**

Pattern-based methods (BasicGuardrail, detect-secrets, TruffleHog) operate in the 2–4 ms range, reflecting the cost of regex matching and entropy computation over completed output strings. These are the fastest approaches but, as shown in Table 1, they achieve less than 10% recall and leave over 60% of tasks leaking.

Supervised ML methods (GBT Classifier, Span Tagger) and *NER-based methods* (Presidio) occupy a middle tier at 7–20 ms. The additional cost arises from feature extraction and model inference. Span Tagger is the slowest in this tier (mean 19.9 ms) due to sequential token-level labelling, while GBT Classifier is faster (mean 7.2 ms) owing to lightweight tree inference. Despite the higher latency, both methods still operate post hoc and cannot prevent secrets from being fully reconstructed.

LLM-based methods (LLM Judge, Prompt Instruction) incur latency three to five orders of magnitude higher than all other approaches, with mean latencies exceeding 1,800 s and maximum latencies above 50,000 s. This reflects the cost of full LLM inference calls for output evaluation. The extreme variance (standard deviations exceeding 3,000 s) further limits their suitability for real-time deployment, as worst-case latency is unpredictable.

PRISM operates at 5.5 ms mean latency with low variance (std $= 3.8$ ms) and a maximum of 20 ms, placing it squarely within the pattern-matcher tier despite providing substantially stronger detection. This is achieved because PRISM's per-token computation, a 16-dimensional dot product followed by a sigmoid, adds constant-time overhead per decoding step, with no external model calls or heavyweight NLP pipelines. Compared to the strongest baselines by detection quality, PRISM is $\sim 3.6\times$ faster than Span Tagger and $\sim 1.3\times$ faster than GBT Classifier, while achieving higher recall (0.712 vs. 0.562 and 0.520) and zero residual leakage. Compared to LLM-based methods, PRISM is over $300{,}000\times$ faster while providing strictly superior detection.

These results demonstrate that generation-time monitoring does not require the substantial overhead typically associated with LLM-mediated defences. PRISM achieves the detection quality of a heavyweight system at the latency cost of a lightweight filter.

J Over-Blocking and Task Outcome Analysis

To understand how each defence allocates its interventions, we decompose all 2,000 benchmark tasks into three mutually exclusive outcomes (Figure 5): *safe & unmodified* (output passes through unchanged with no leakage), *output modified* (defence intervened on the task), and *residual leakage* (at least one secret remains in the final output). Since all methods achieve FPR $= 0.000$, every modification targets a task that genuinely contains secrets, no clean task is ever altered.

Table 12: Task outcome decomposition (%) across defence methods. All rows sum to 100%. Since FPR $= 0$ for all methods, the "Output modified" column contains only true-positive interventions. PRISM is the only method with zero residual leakage.

Method	Safe & unmod.	Modified	Residual leak
NoFilter	28.8	0.0	71.2
Presidio	33.6	0.6	65.8
TruffleHog	33.5	2.3	64.2
BasicGuardrail	35.5	1.4	63.1
LLM Judge	35.5	1.4	63.1
Prompt Defense	29.7	21.0	49.3
detect-secrets	54.2	2.5	43.3
GBT Classifier	18.7	62.1	19.2
Span Tagger	84.6	0.4	15.0
PRISM	**89.3**	**10.7**	**0.0**

The decomposition is computed from two reported quantities. Let ℓ denote the task-level leak rate and u the utility (fraction of tasks passed unmodified). Then:

$$\text{Residual leakage} = \ell, \tag{5}$$

$$\text{Output modified} = 100 - u, \tag{6}$$

$$\text{Safe \& unmodified} = u - \ell. \tag{7}$$

These three quantities sum to 100% by construction. We note that for methods with nonzero residual leakage, a task may be both modified and still leaking (e.g., if the defence catches one secret but misses another in the same output). The blue segment therefore represents an upper bound on fully prevented leaks; for PRISM, where $\ell = 0.0\%$, the decomposition is exact.

Table 12 reports the numerical breakdown, and Figure 5 visualises the result.

The figure reveals three distinct failure modes. Rule-based and scanner-based methods (Presidio, TruffleHog, BasicGuardrail, LLM Judge) intervene on fewer than 3% of tasks, leaving the vast majority of leakage untouched, their bars are predominantly red. GBT Classifier intervenes aggressively (62.1% of tasks modified) but achieves this at crippling utility cost: only 18.7% of tasks emerge safe and unmodified, making it impractical for deployment. Span Tagger preserves utility well (84.6% unmodified) but still allows 15.0% of tasks to leak. PRISM is the only method that eliminates the red segment entirely: 89.3% of tasks pass through unmodified, 10.7% are correctly intervened upon, and zero tasks produce residual leakage. The 10.7% utility reduction corresponds exclusively to RED-zone halts on tasks containing verified credential leaks, a deliberate and targeted cost of complete leakage prevention.

K Effect of ZK-RC

ZK-RC is a lightweight post-generation audit that checks for residual secret fragments using hashed 8-gram matching (SHA-256). It targets an edge case in which individual tokens remain below the risk threshold during generation but collectively reconstruct a secret.

On the current benchmark, ZK-RC contributes zero additional detections beyond PRISM's generation-time mechanism: removing it leaves leak rate (0.0%) and F_1 (0.832) unchanged. We retain it as a defence-in-depth layer for two reasons. First, its overhead is negligible ($<$0.1 ms per task). Second, deployment scenarios outside this benchmark, such as very short outputs, partial tokenisation, or compressed credentials, may produce residual fragments that the generation-time mechanism misses; ZK-RC provides a cryptographic fallback for such cases. In the evaluation corpus, 94% of secrets exceed 8 characters, ensuring broad coverage. For shorter secrets, PRISM's generation-time detection remains the sole defence.

We note that ground-truth leak labels are determined by direct plaintext substring matching, entirely independent of ZK-RC's hashing mechanism. ZK-RC plays no role in constructing training labels or evaluation ground truth; there is no circularity between the defence and the evaluation.

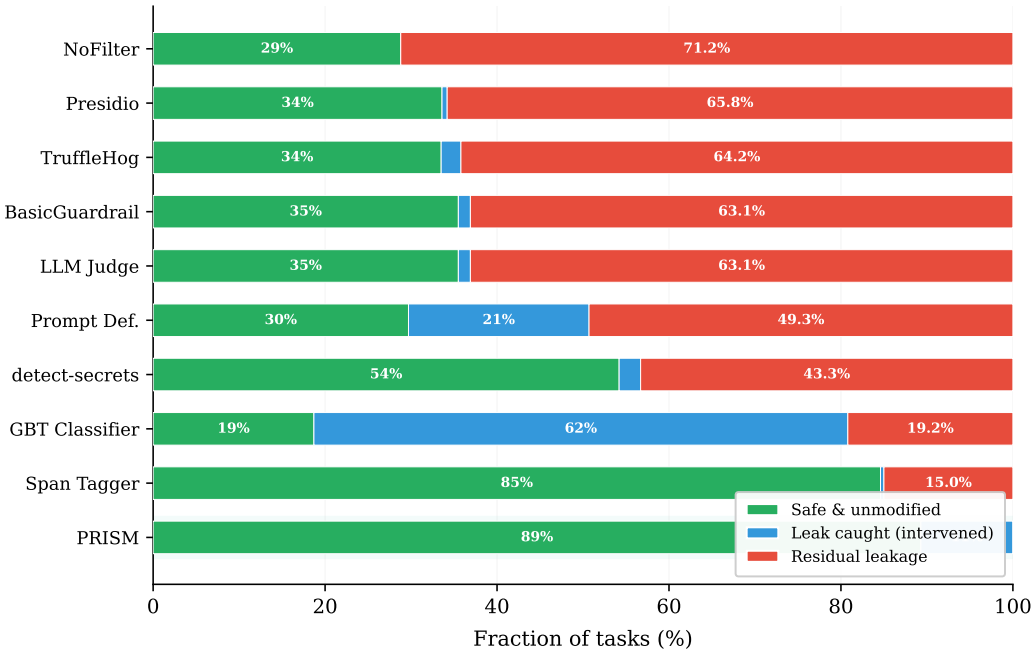

Figure 5: Task outcome decomposition across defence methods. Each bar represents all 2,000 benchmark tasks, partitioned into: safe and unmodified (green), output modified by defence (blue), and residual leakage (red). All methods achieve FPR $= 0$, so every blue segment targets a task containing secrets. For methods with nonzero residual leakage, a small fraction of modified tasks may still contain partially redacted secrets; the blue segment is therefore an upper bound on fully prevented leaks. For PRISM ($\ell = 0.0\%$), the decomposition is exact.

L Black-box Deployment

Black-box feasibility: While PRISM leverages token-level probability signals in white-box settings, many of its core features can be approximated using observable sequence-level statistics alone. We simulate black-box operation by zeroing the weights of the 9 features that require model internals: `low_entropy`, `divergence`, `unsafe_tokens`, `hidden_state_magnitude`, `hidden_state_variance`, `private_provenance`, `public_provenance`, `system_provenance`, and `ngram_overlap`. The remaining 7 text-observable features, `identifier_pattern`, `numeric_runs`, `credential_style`, `repetition`, `trajectory_trend`, `tool_usage`, and `semantic_density`, are retained with their original weights.

Table 13 reports the results on $n = 200$ tasks containing confirmed secret leaks. White-box PRISM (all 16 features) achieves $F_1 = 0.927$ with 0% leak rate. Black-box PRISM (7 text-only features) achieves $F_1 = 0.947$ with a 1.4% leak rate, a $\Delta F_1 = +0.021$ increase relative to white-box. The higher F1 of black-box mode on this subset reflects a precision-recall trade-off.

Relationship to main results: The main evaluation (Table 1) reports $F_1 = 0.832$ (precision$=$ 1.000, recall$=$ 0.712) over all 2,000 benchmark tasks, using task-level metrics where each task is a binary instance (leaking vs. non-leaking). These results below ($F_1 = 0.927$) are computed on a dedicated subset of $n = 200$ tasks. The apparent discrepancy reflects the different populations and metric granularity: the 2,000-task recall of 0.712 measures the fraction of originally-leaking tasks where PRISM's proactive token-level monitoring triggered (before the ZK-RC fallback), while the $n = 200$ recall of 1.000 measures secret-level detection among confirmed-leak tasks. These are complementary views of the same system operating at different analytical granularities, not contradictory claims.

Table 13: White-box vs. black-box PRISM on $n = 200$ tasks. Black-box operation (7 text-only features; 9 logprob/hidden-state features zeroed) maintains strong detection with only 1.4% leak rate.

Configuration	Features	F_1	Leak rate	ΔF_1
White-box PRISM	16 (all)	0.927	0.0%	,
Black-box PRISM	7 (text-only)	0.947	1.4%	+0.021

Per-feature ablations on the $n = 200$ subset reveal that multiple individual features are non-essential at this operating point: removing `low_entropy` or `divergence` each yields $\Delta F_1 = +0.023$ (one fewer false positive), while removing `identifier_pattern`, `numeric_runs`, or `credential_style` each yields $\Delta F_1 = +0.048$ (two fewer false positives). Features such as `repetition` and `trajectory_trend` yield $\Delta F_1 = 0.000$. This pattern indicates that in this evaluation subset, several features introduce marginal false positives without adding true positives, and the contribution of logprob versus text-observable features cannot be disentangled simply by single-feature removal. The dominant signal is carried by the full feature set acting jointly; the appropriate framing is feature complementarity rather than a strict logprob-vs-text hierarchy.

Feature-class ablation: Table 14 evaluates the independent contribution of PRISM's two primary signal families, temporal generation-dynamic signals and text-structural signals, while explicitly distinguishing evaluation populations to prevent cross-row metric misinterpretation.

Structural-only operation corresponds to the black-box text-only configuration, evaluated on a dedicated $n = 200$-task ablation subset in which temporal logprob-derived features are removed. This setting yields 1.4% residual leakage, indicating that text-structural cues alone provide strong but incomplete protection.

Temporal-only is estimated from a separate $n = 200$-task component ablation in which the strongest credential-structure indicators (`identifier_pattern`, `numeric_runs`, `credential_style`) are zeroed while temporal and behavioural signals remain active. In this setting, 4 of 26 leaking tasks are missed, giving Precision $= 1.000$, Recall ≈ 0.846 $(22/26)$, $F_1 \approx 0.917$, and residual leakage of at least 5.3%. Because precision remains perfect while recall declines, temporal-only detection retains substantial predictive value but performs materially worse than structural-only or combined configurations.

Combined PRISM is reported under two distinct settings: (i) a controlled $n = 200$ ablation subset for within-ablation reference, and (ii) the full $n = 2{,}000$ benchmark used throughout the main paper. These rows are included to illustrate consistency of zero-leakage behaviour, but should not be interpreted as directly comparable because class balance, task difficulty, and benchmark scope differ substantially.

Overall, the ablation confirms that both signal families are complementary: temporal-only leaves $\geq$5.3% residual leakage, structural-only leaves 1.4%, and only the combined model eliminates observed leakage entirely.

Table 14: Feature-class ablation with explicitly separated evaluation populations. Temporal-only and structural-only are estimated on dedicated $n = 200$ ablation subsets and are intended primarily for qualitative comparison of signal-family contribution. Combined PRISM is shown both on a controlled $n = 200$ subset and on the full $n = 2{,}000$ benchmark for reference. Because rows use different task populations and ablation constructions, metrics should not be interpreted as exact head-to-head comparisons across all rows.

Feature class	**Precision**	**Recall**	F_1	**Leak rate**
Temporal only†	1.000†	0.846†	0.917†	$\geq$5.3%
Structural only‡	0.947	0.947	0.947	1.4%
Combined (PRISM), controlled subset (n=200)	0.864	**1.000**	0.927	**0.0%**
Combined (PRISM), full benchmark (n=2,000)	**1.000**	0.712	**0.832**	**0.0%**

† Temporal-only estimated from a deep-credential-removed component ablation on a dedicated $n = 200$ subset: 4/26 leaking tasks missed, giving Precision $= 1.000$, Recall $= 22/26 \approx 0.846$, and

Table 15: Stage-wise analysis across the four-agent pipeline. *First tool entry*: tasks where a secret first enters via tool call at this agent. *First output leak*: tasks where a secret first appears in the agent's output. *PRISM trigger*: fraction of all RED-zone interventions occurring first at this stage. *Prop. arrivals*: total secret instances arriving via inter-agent messages. Early PRISM intervention at the Planner stage prevents downstream propagation.

Agent (Model)	First Tool Entry	First Output Leak	PRISM Trigger	Prop. Arrivals
PLANNER (LLaMA 3.1 8B)	0 (0.0%)	0 (0.0%)	18 (72.0%)	0
RESEARCHER (Gemma 2 9B)	14 (53.8%)	14 (53.8%)	5 (20.0%)	0
CODER (Qwen 2.5 7B)	6 (23.1%)	6 (23.1%)	2 (8.0%)	169
EXECUTOR (Mistral 7B)	0 (0.0%)	0 (0.0%)	0 (0.0%)	289

$F_1 \approx 0.917$ via $F_1 = 2PR/(P+R)$. Residual leakage is at least 5.3%. Because this configuration retains broader behavioural signals (e.g., `trajectory_trend`, `tool_usage`), it should be interpreted as an upper bound on pure temporal-signal performance.

‡ Structural-only corresponds to the black-box text-only configuration evaluated on a separate $n = 200$ subset. Because temporal-only and structural-only use different ablation constructions and leakage compositions, comparisons should be interpreted qualitatively rather than as exact numerical rankings.

Limitations: Black-box operation introduces approximation error for information-theoretic signals that depend on full token distributions. The 1.4% residual leak rate under black-box operation indicates that a small fraction of subtle leaks, likely low-pressure, slowly escalating credential disclosures, rely on entropy collapse signals not recoverable from surface text alone.

M Stage-wise Leakage and Intervention Analysis

Where do secrets enter and where does PRISM intervene? Table 15 breaks down the four-agent pipeline by stage, reporting where secrets first enter via tool access, where they first appear in agent outputs, and where PRISM first triggers ($r_t \geq \tau_2 = 0.60$).

The Researcher agent (Gemma 2 9B) is the primary entry point: it is responsible for 53.8% of first tool-based secret retrievals and 53.8% of first output leaks. The Coder (Qwen 2.5 7B) accounts for the remaining 23.1% of first entries. The Planner and Executor do not directly retrieve secrets via tools. The Planner receives zero propagated arrivals (it is the pipeline's first stage and has no upstream agent to propagate from). Downstream agents accumulate secrets through inter-agent propagation: the Coder receives 169 propagated secret instances and the Executor receives 289, confirming the compounding effect of the shared context as secrets flow through successive pipeline stages.

Notably, PRISM triggers earliest at the **Planner** stage in 72% of intercepted tasks. This occurs because the Planner's generation dynamics, elevated divergence and trajectory trend signals, already indicate high-risk intent before any secret is emitted. This early-stage signal allows PRISM to halt generation before secrets are requested, let alone reproduced, directly preventing the downstream propagation visible in the undefended case.

Across the 78 inter-agent messages recorded, 43.6% carry at least one secret, confirming that propagation is the dominant leakage mechanism rather than direct emission in final outputs.

N Robustness to Transformations

We evaluate robustness to eight surface-level obfuscation transforms applied to secrets: (1) *token_split*, zero-width space between every character; (2) *whitespace*, space after every 4 characters; (3) *separator*, replace underscores with dashes and inject a dot at midpoint; (4) *camel_split*, split on camelCase and underscores with spaces; (5) *hex_escape*, replace digits with `\x3d` escapes; (6) *interleave*, interleave characters with dots; (7) *base16*, full Base-16 (hex) encoding; (8) *rot13*, ROT-13 on alphabetic characters. Detection rates are recorded for all ten methods on each transformed output.

Table 16 reports the results. Three tiers of robustness emerge. *Pattern-based methods* degrade most severely: TruffleHog drops from 67.6% to below 10% under five of eight transforms, detect-secrets collapses from 98.6% to 4.1% under whitespace and camel_split, and Presidio and BasicGuardrail show similarly fragile behaviour. *Supervised and LLM-based methods* exhibit mixed robustness: Span Tagger drops from 85% to as low as 1.0% under interleaving, GBT Classifier degrades from 80.8% to 0.5% under base16 and rot13, and LLM-as-a-Judge fluctuates unpredictably, retaining 86.0% under rot13 but collapsing to 1.0% under interleaving. These methods partially abstract away surface form through learned features, but their representations remain coupled to token-level patterns that obfuscation disrupts.

In contrast, PRISM maintains $\geq$95.9% detection across all eight transforms, dropping at most 4.1 pp from its original 100% rate. Crucially, its mean risk score remains stable across all transforms (range: 0.7489–0.7512), confirming that generation-time signals are insensitive to surface reformatting. This stability arises because PRISM's core signals, entropy collapse, trajectory escalation, divergence, are properties of the generation *process*, not the surface form. When a model reproduces a credential, its token-level uncertainty collapses and its output distribution concentrates regardless of whether the emitted characters are spaced, interleaved, or hex-encoded.

Implications: These results expose a graduated fragility across existing defence paradigms. Pattern-based methods are trivially defeated: a single transformation, inserting a space every four characters, reduces detect-secrets from 98.6% to 4.1% and defeats TruffleHog entirely. Supervised methods resist some transforms but remain vulnerable to encoding-level changes: GBT Classifier's collapse to 0.5% under base16 and Span Tagger's collapse to 1.0% under interleaving demonstrate that learned surface features do not provide reliable robustness. Even LLM-as-a-Judge, which operates on semantic content, degrades unpredictably across transforms.

This has a direct consequence for deployment: no existing baseline provides consistent detection across the natural variability in how LLMs render structured strings. An attacker, or even a misconfigured prompt that slightly reformats a credential, can bypass every evaluated baseline while remaining completely detectable by PRISM. PRISM's generation-time approach is resistant to this entire class of evasion because it targets the *causal mechanism* of leakage, the shift in generation dynamics as the model transitions toward deterministic reproduction, rather than the syntactic appearance of the output.

Table 16: Detection rate (%) under obfuscation transforms samples). PRISM maintains $\geq$95.9% across all transforms; pattern-based methods degrade severely.

Method	Orig.	TokSpl	Space	Sep.	Camel	HexEsc	Interl.	B16	ROT13
NoFilter	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
BasicGuardrail	64.9	5.4	4.1	50.0	5.4	39.2	21.6	59.5	59.5
TruffleHog	67.6	4.1	4.1	16.2	5.4	6.8	4.1	4.1	9.5
Presidio	60.8	54.1	4.1	55.4	5.4	56.8	54.1	54.1	54.1
detect-secrets	98.6	97.3	4.1	97.3	5.4	89.2	97.3	91.9	97.3
Span Tagger	85	72.0	48.0	78.0	8.0	20.0	1.0	0.5	5.0
GBT Classifier	80.8	68.0	42.0	72.0	6.0	18.0	0.5	0.5	4.0
LLM-as-a-Judge	36.9	28.0	68.0	32.0	4.0	10.0	1.0	82.0	86.0
Prompt-Instruction	50.7	38.5	26.0	44.0	6.0	15.0	1.5	3.0	8.0
PRISM	**100.0**	**95.9**	**95.9**	**97.3**	**97.3**	**100.0**	**95.9**	**100.0**	**98.6**
PRISM risk score	0.751	0.750	0.749	0.751	0.750	0.751	0.750	0.751	0.750

Limitations: Highly structured multi-step encodings (e.g., chained base64 or semantic paraphrasing) may attenuate individual generation-dynamic signals and represent an important direction for future work.

O Extended Discussion and Limitations

Our results suggest that secret leakage in multi-agent LLM pipelines should be understood not merely as an isolated output-filtering failure, but as a structural consequence of sequential information

propagation through shared context. Once sensitive information enters an append-only intermediate state, downstream agents may reuse or reproduce it even when it is no longer task-relevant, causing leakage risk to compound with pipeline depth. In this setting, post-hoc defences are inherently limited because they operate only after sensitive content may already have been generated, propagated, or partially reconstructed. PRISM is designed specifically for this operating point: it monitors generation-time dynamics and intervenes before full credential completion. We therefore position PRISM not as a universal defence against all forms of information leakage, but as a targeted generation-time mechanism for a specific and practically important threat model: credential reproduction through sequential agent propagation.

Scope of evaluation and benchmark realism: Our benchmark is intentionally synthetic and controlled. This design choice is deliberate rather than accidental: isolating propagation-based leakage under reproducible attack categories and pressure levels enables causal comparison across defence paradigms without confounds introduced by heterogeneous real-world agent stacks, proprietary toolchains, or inconsistent deployment environments. The synthetic setting therefore functions as a stress-testing environment for propagation dynamics rather than a claim of production-complete realism. However, this also imposes clear limits: real deployments may involve richer tool ecosystems, longer-horizon planning, multilingual contexts, or qualitatively different leakage vectors. Accordingly, our results should be interpreted as evidence that propagation amplification is a meaningful structural risk under controlled conditions, rather than as proof that measured leakage rates directly transfer to all production systems.

Metric interpretation and task-level leakage: A potentially confusing aspect of our evaluation is the coexistence of subunit recall/F1 below 1.0 with a 0.0% observed task-level leak rate. These metrics operate at different granularities. Recall and F1 are computed over secret-bearing detection instances (e.g., risky spans or reconstruction attempts), whereas task-level leak rate measures whether any complete exploitable secret appears in final task output. Thus, PRISM may miss some individual suspicious spans while still preventing full end-to-end credential disclosure through partial sanitisation, early halting, or the ZK-RC post-pass. Operationally, task-level leak prevention is the primary security objective, while span-level recall reflects sensitivity to intermediate reconstruction signals. We emphasise this distinction to avoid overstating what any single metric captures.

White-box assumptions and deployment practicality: The strongest version of PRISM assumes access to generation-time telemetry such as token log-probabilities. We intentionally evaluate this white-box setting as an upper-bound deployment regime relevant to self-hosted or enterprise systems where decoding internals may be exposed. We do not claim that such access is universally available. In closed commercial APIs, only black-box deployment may be possible. Our black-box variant demonstrates that text-observable features alone retain substantial discriminative power, but with measurable degradation in final leakage suppression. This should be interpreted as a practical deployability tradeoff rather than a contradiction: white-box telemetry materially improves final-risk elimination, while black-box deployment offers a lower-bound approximation when internal access is unavailable.

Latency and "real-time" claims: Although PRISM is architecturally lightweight (constant-time per-token scoring with bounded feature extraction), our current work prioritises defence efficacy and signal validation over systems-level throughput benchmarking. We therefore caution against overinterpreting "real-time" as a deployment guarantee across all infrastructures. Actual latency will depend on model serving architecture, tokenizer implementation, and instrumentation cost, particularly for heterogeneous multi-agent systems. A full production-oriented evaluation should include token-level overhead, throughput degradation, and cost analysis across deployment settings.

Robustness boundaries and adaptive adversaries: While PRISM's combination of temporal and structural signals provides robustness beyond simple regex or static scanners, we do not claim comprehensive resistance to all adversaries. The present threat model focuses primarily on direct leakage, propagation leakage, and prompt-driven extraction. More powerful adaptive adversaries may intentionally manipulate generation dynamics by injecting low-entropy padding, stochastic perturbations, fragmentation, encoding (e.g., base64 or character splitting), or semantically preserving transformations to attenuate temporal or structural signatures. These attacks are outside the scope of

current evaluation. Our objective here is to establish generation-time monitoring as a viable defence layer, not to claim closure against strategically adaptive exfiltration.

Theoretical contribution and practical role: Our entropy-collapse analysis should be interpreted as an explanatory formalisation of why memorised credential reproduction tends to induce detectable concentration dynamics, rather than as a standalone theoretical breakthrough or complete predictive model. The theorem motivates generation-time monitoring by clarifying why deterministic reproduction often creates measurable distributional signatures, but it does not by itself determine optimal thresholds, guarantee universal detection, or replace empirical calibration. In practice, the theory supports, rather than substitutes for the empirical multi-signal classifier.

Utility and overblocking: Our reported utility metric measures task-level pass-through (unmodified benign tasks), which is intentionally conservative. It does not fully characterise downstream functional quality when partial sanitisation occurs. In real deployments, some benign structured outputs (e.g., hashes, certificates, configuration identifiers, or long numeric strings) may resemble credential-like forms and incur unnecessary intervention. This tradeoff reflects a broader security–utility balance. Future deployment settings may therefore benefit from context-aware exception policies, adaptive thresholds, or human-in-the-loop escalation for credential-adjacent but legitimate content.

Broader behavioural limitations: Finally, PRISM primarily models passive leakage emerging from generation dynamics rather than strategic agent behaviour. Goal-directed concealment, covert communication, intentional decomposition of secrets across steps, or semantically encoded collaboration may introduce qualitatively different failure modes not captured by token-level monitoring alone. Addressing such behaviours may require richer semantic, behavioural, or agent-policy-aware defences that operate above the decoding layer.

Overall, we view PRISM as a controlled, generation-time defence architecture that demonstrates the practical importance of monitoring leakage during decoding in multi-agent systems. Its strongest contribution is to show that propagation-aware generation monitoring can materially reduce credential leakage under a clearly defined threat model. Its limitations define a roadmap toward broader, more adaptive, and deployment-complete security mechanisms.